

User Guide - Cognizant® Code Companion Prompt Library

Table of Contents

Introduction	3
Pre-requisites.....	3
Prompt Library License	3
Steps to fetch UUID	3
Installation Steps	4
VS Code	4
IntelliJ.....	5
Visual Studio	6
Steps to use Cognizant Code Companion Prompt Library.....	7
VS Code	8
IntelliJ.....	9
Visual Studio	11
Customization of Cognizant Code Companion Prompt Library	12
VS Code	12
IntelliJ.....	14
Visual Studio	17
Addition of new custom prompts.....	20
Logging Prompt Library Usage Metrics	22
Introduction.....	22
Prompt Usage Logging in local and report visualization.....	22
Prompt Usage Logging as JSON file.....	22
Prompt Usage Logging using Open Telemetry	22
Steps to enable Open Telemetry Settings.....	23
VS Code	23
IntelliJ.....	24
Open Telemetry Installation and Configuration	24
Prompt Library Integration in GitHub Copilot Chat	25

BRE Integration in GitHub Copilot Chat	27
Configuration for BRE Extraction.....	27
Steps for using BRE Extraction in GHCP	29
Prompt Library Logs.....	31
Prompt Usage JSON Path	31
Troubleshooting	32
User is receiving a notification: "License not found, please contact your Admin"	32
User is not receiving notification: "Prompts for GitHub Copilot is active", post installation of Prompt Library extension.....	32
Prompt Usage JSON is not getting updated	32

Introduction

Cognizant has developed a customizable prompt library to enhance developer productivity.

- The Cognizant® Code Companion Prompt Library is a plugin, developers can install the plugin in VS Code, IntelliJ, Visual Studio and Android Studio.
- Developers can use this plugin on top of the code companions Copilot, Amazon Q, Gemini and Tabnine to improve their productivity.
- The plugin contains Categories, subcategories for prompts. A category is a broad grouping of related prompts, while a subcategory is a more specific grouping within a category. After selecting a category and sub-category, the corresponding prompt will be loaded into the developer's IDE. This allows the developer to use the prompt directly in the code companion, enhancing productivity by eliminating the need to write the prompt manually.
- Repeated prompts can be added to the library and reused across all pods within the engagement.
- Plugin supports multiple languages and can be tested with Java, .NET, Cobol, Angular, React, and NodeJS.

Pre-requisites

- JDK8 or higher (till JDK17)
- Prompt library license.
- Cognizant® Code Companion Prompt Library plugin for the developer specific IDE.

Prompt Library License

Prompt library license is required to use Cognizant® Code Companion Prompt Library plugin.

Please refer to the below steps for sharing UUID details to generate license for specific OS:

Steps to fetch UUID

Windows OS

Open the Windows PowerShell (**Win + X** then press **I**) and run the below command to get UUID,

- Command: **Get-CimInstance -Class Win32_ComputerSystemProduct | Select-Object -ExpandProperty UUID**
- Share the below highlighted “UUID” to the product team.

```
PS C:\Users\842242> Get-CimInstance -Class Win32_ComputerSystemProduct | Select-Object -ExpandProperty UUID  
4C4C4544-0037-3710-8046-B1C04F533634
```

MAC OS

Open the command prompt and run the below command to get UUID,

- Command: **system_profiler SPHardwareDataType | grep "Hardware UUID"**
- Share the below highlighted "UUID" to the product team

```
901362@AMB00680 ~ % system_profiler SPHardwareDataType | grep "Hardware UUID"  
Hardware UUID: 7C05BC1E-1F50-5F71-BE8B-48BFC63EA3ED
```

Once license file is generated, rename it to offline.lic and copy it in user home directory

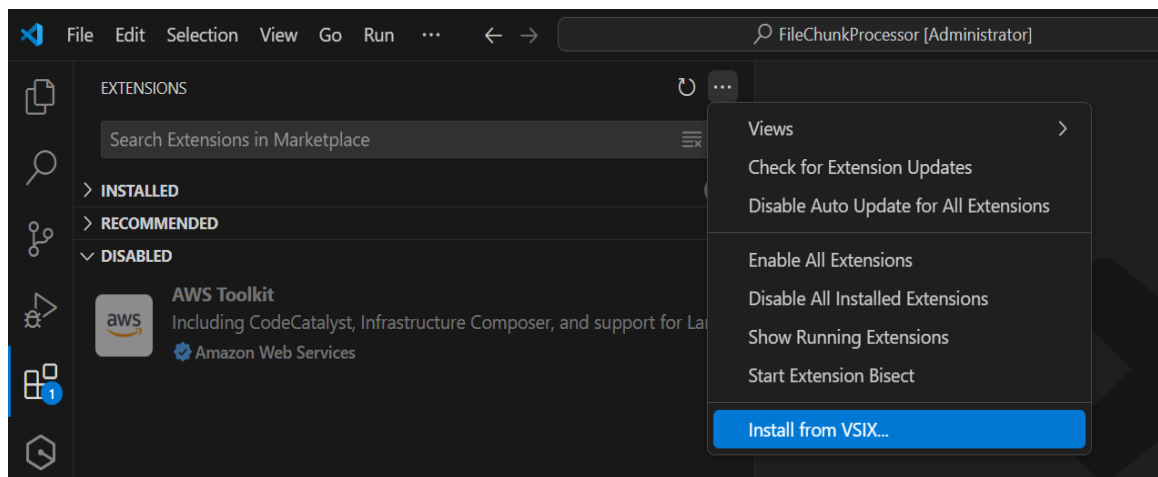
e.g. For Windows, it should be copied in "C:\\Users\\2363724 "

For MAC, it should be copied in "/Users/2363724 "

Installation Steps

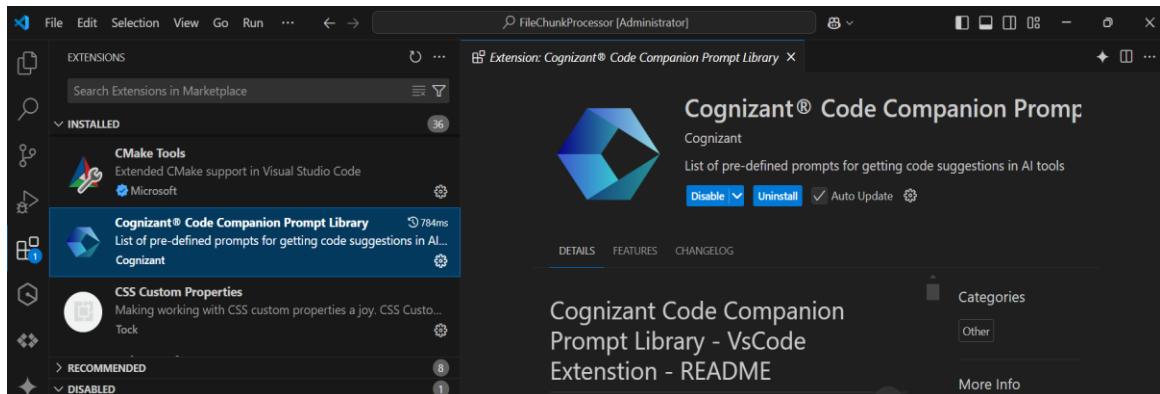
VS Code

1. Open Visual Studio Code.
2. Go to the Extensions (or press Cmd+Shift+X on Mac, Ctrl+Shift+X on Windows).
3. Click on the ellipsis icon (...) and then select the option "**Install from VSIX**".

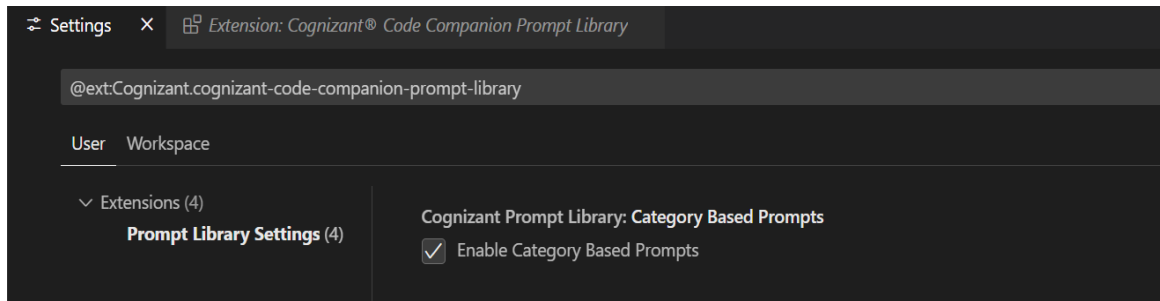


4. Navigate to the directory where you have placed the VSIX file received. Select the plugin and install it.

- After successful installation, the plugin will appear in the list of installed extensions.

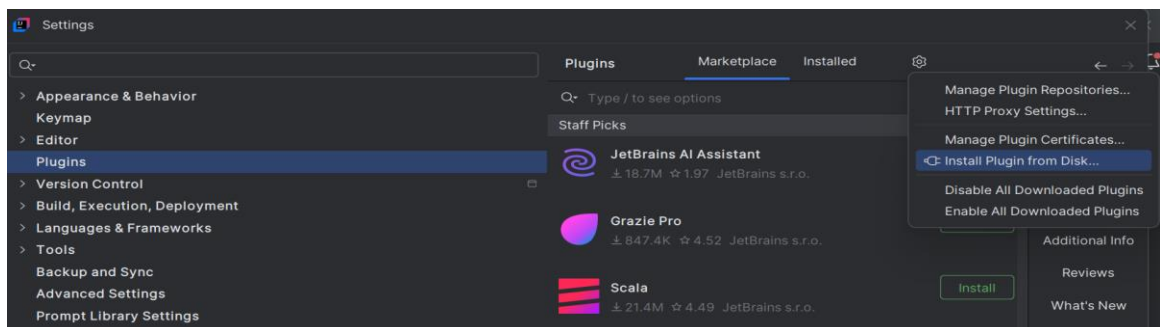


- Click on the settings icon and then select "Extension Settings" and ensure "Enable Category Based Prompts" checkbox is enabled.



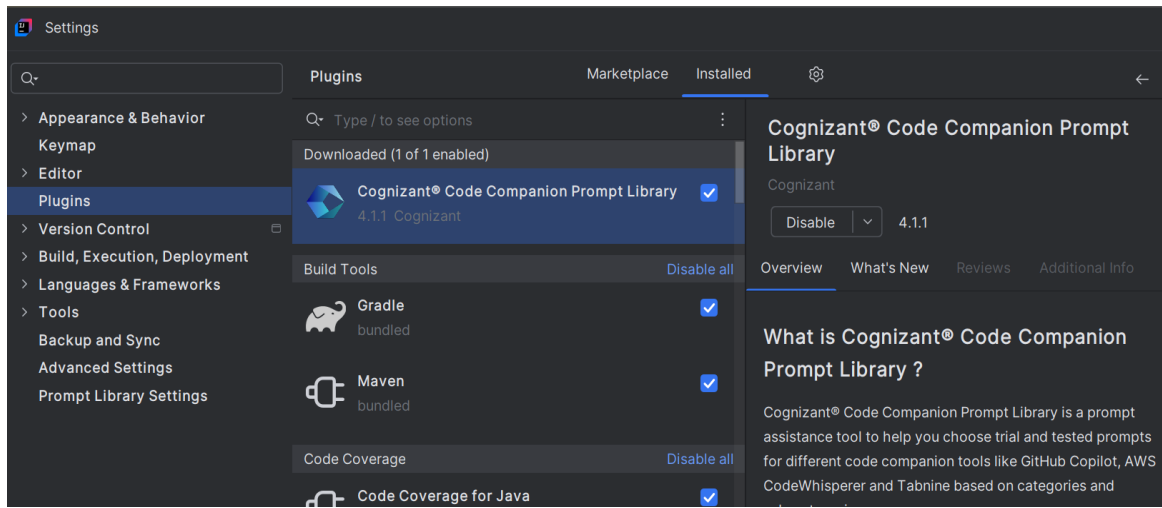
IntelliJ

- Open IntelliJ IDEA.
- Go to File > Settings > Plugins
- Click on the settings icon next to Installed tab and select the option "Install Plugin from Disk".



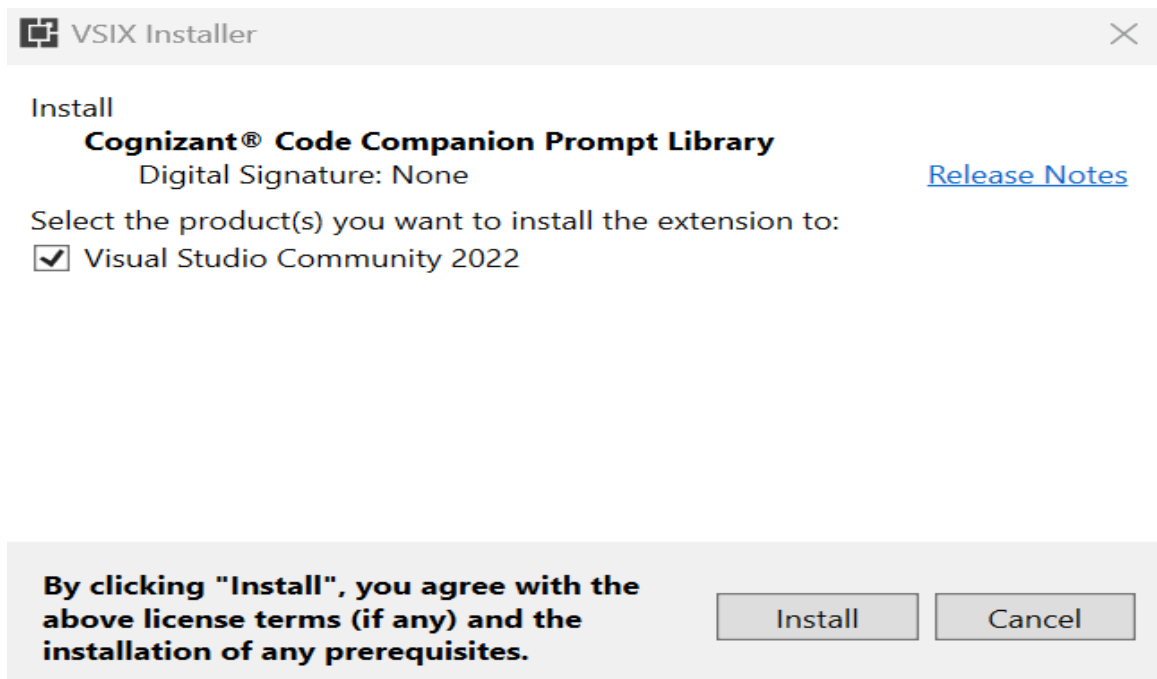
- Navigate to the directory where you have placed the jar file. Select the plugin and install it.

5. After successful installation, the plugin will appear in the list of installed extensions.

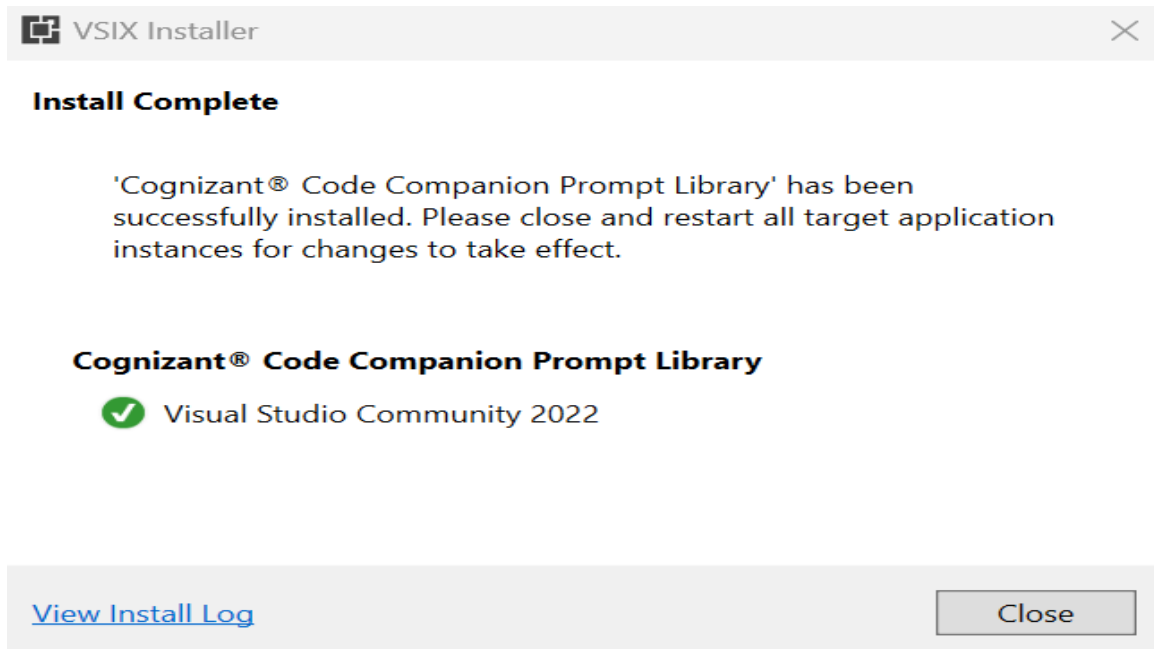


Visual Studio

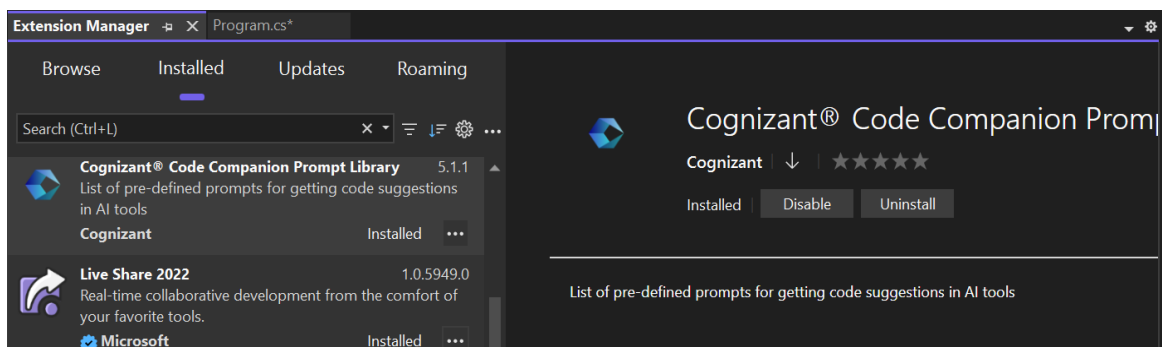
1. Double click **Cognizant Code Companion Prompt Library VSIX** file, which you have received from Cognizant product team.
2. Click the Install button in the popup window.



3. Close the window once installation gets completed.



4. After successful installation, the plugin will appear in the list of installed extensions.



Steps to use Cognizant Code Companion Prompt Library

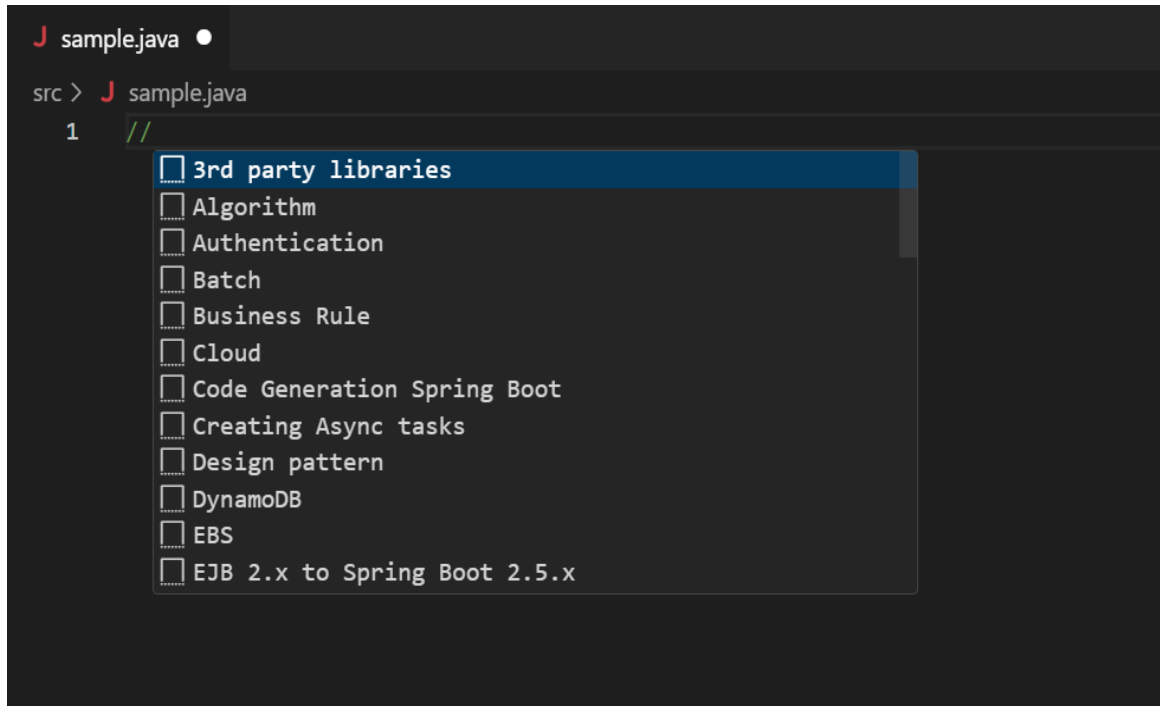
1. Ensure the respective IDE cognizant code companion prompt library plugin is installed.
2. Start typing a comment character based on the file extension to get a dropdown with a list of Prompt Categories.

e.g. ".java", ".cs", ".js" will use "//" to invoke the Cognizant® Code Companion Prompt Library. Other languages like Python, Cobol and markdown files will use "#", etc.
3. Based on the category selection, subcategories will be loaded. Upon selecting a subcategory, the actual prompt will be inserted for the user.

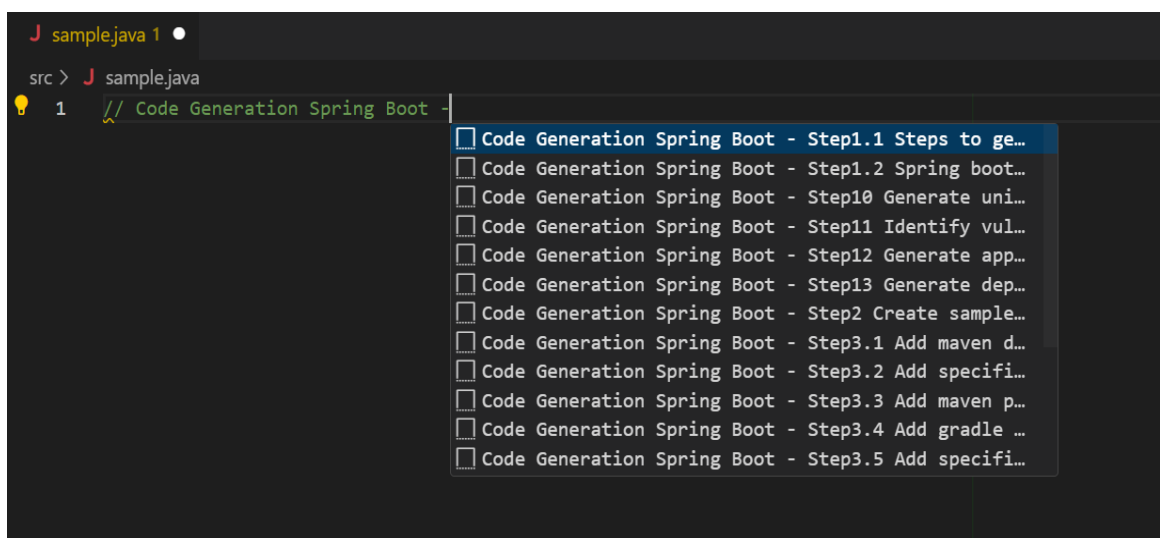
VS Code

Step-by-Step Example for a Java File (.java)

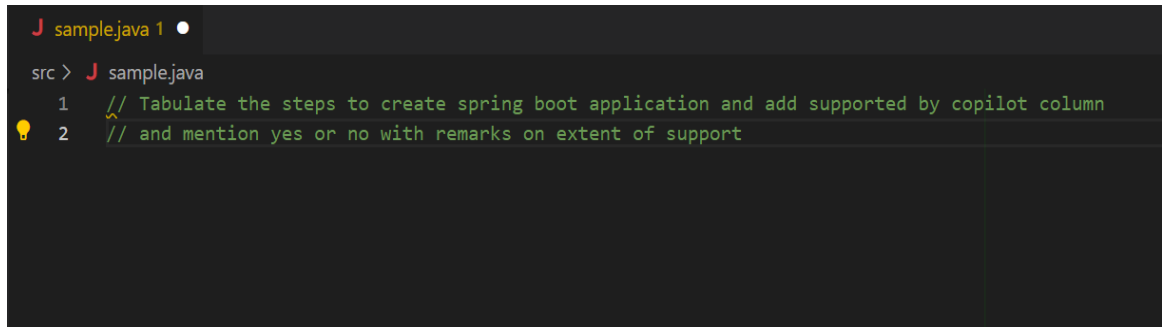
1. After typing comment character “//”, list of prompt categories will be displayed for the specific file type. The user can select the appropriate category from this list.



2. After selecting a category, type a delimiter hyphen (-) to list down the subcategories for the selected category. The user can choose the appropriate subcategory from the list.



3. Once the user selects a subcategory, corresponding prompt will be inserted into the .java file, and the selected category and subcategory text will be automatically removed.

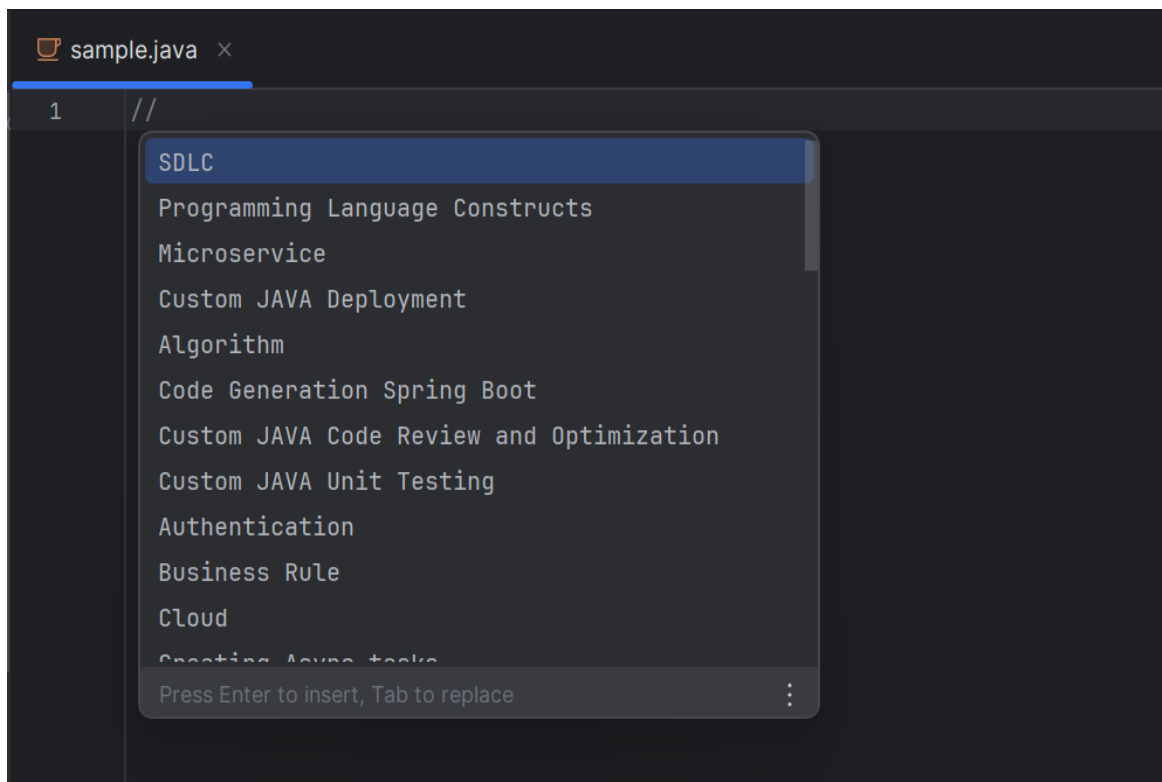


```
J sample.java 1 ●
src > J sample.java
1 // Tabulate the steps to create spring boot application and add supported by copilot column
2 // and mention yes or no with remarks on extent of support
```

IntelliJ

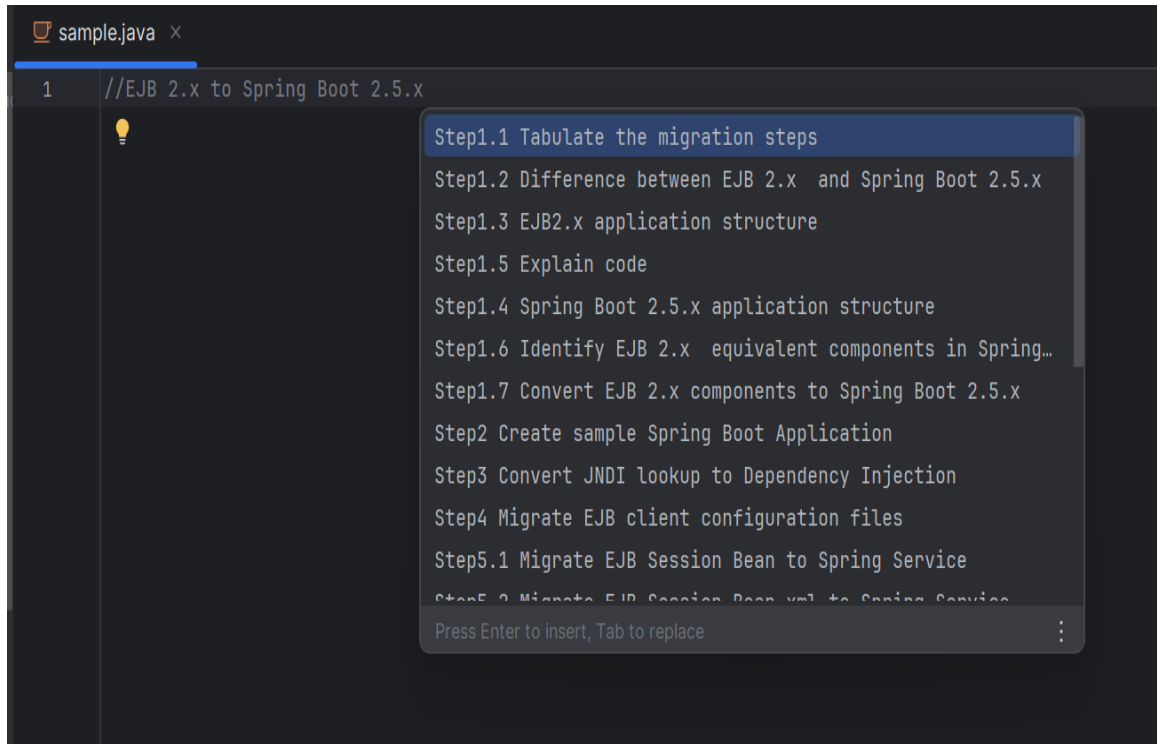
Step-by-Step Example for a Java File (.java)

1. After typing the comment character "("//", press CTRL + Space to list down the prompt categories for the specific file type. The user can select the appropriate category from the list.

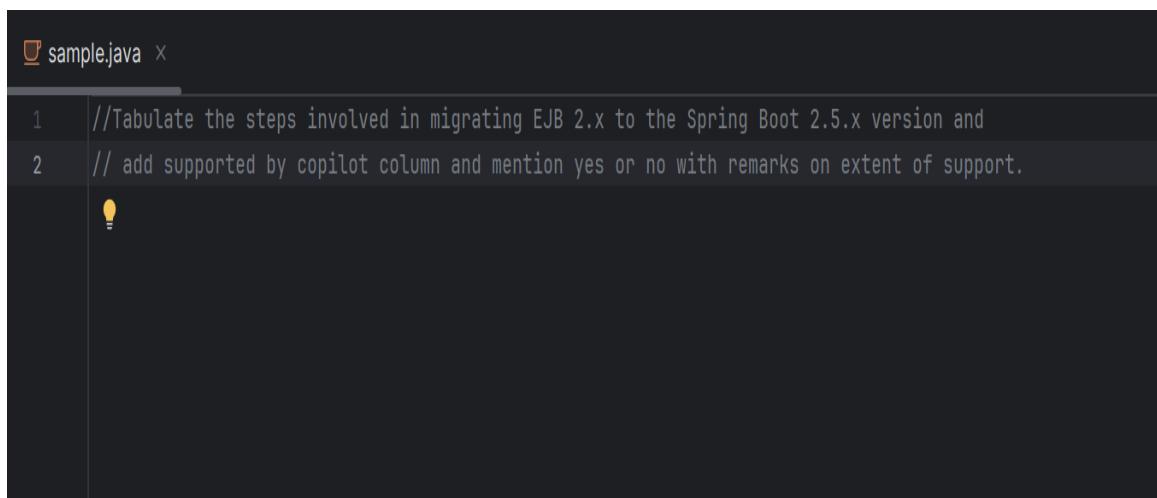


```
sample.java x
1 //
SDLC
Programming Language Constructs
Microservice
Custom JAVA Deployment
Algorithm
Code Generation Spring Boot
Custom JAVA Code Review and Optimization
Custom JAVA Unit Testing
Authentication
Business Rule
Cloud
Creating Java tasks
Press Enter to insert, Tab to replace
```

2. After selecting a category, type a space and press Ctrl + Space to display subcategories based on the selected category. The user can choose the appropriate subcategory from the list.



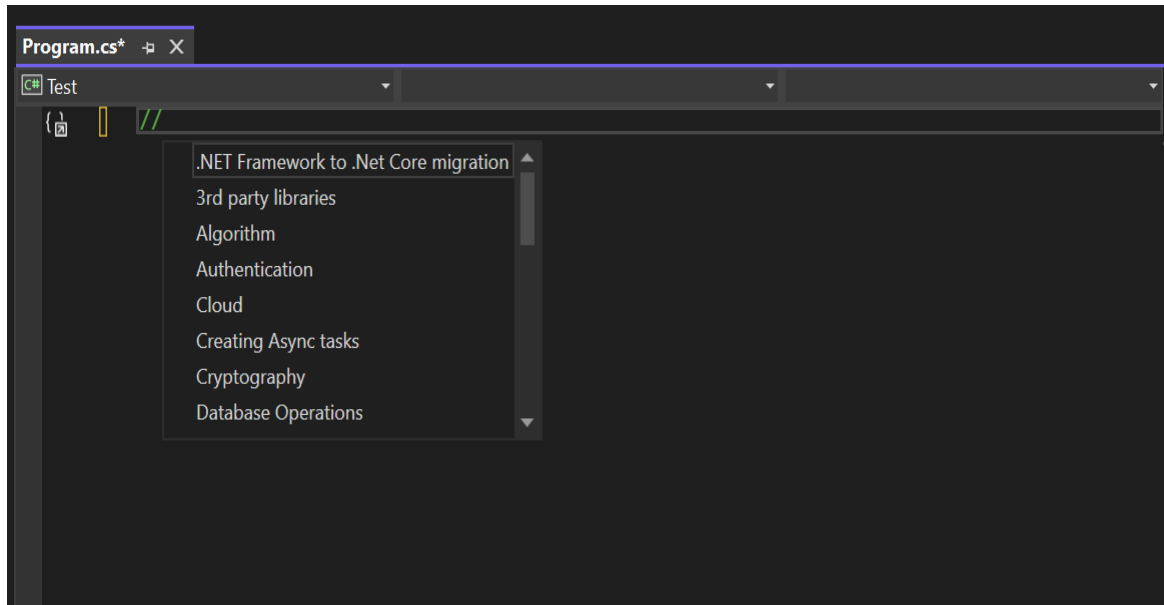
3. Once the user selects a subcategory, corresponding prompt will be inserted into the .java file, and the selected category and subcategory text will be automatically removed.



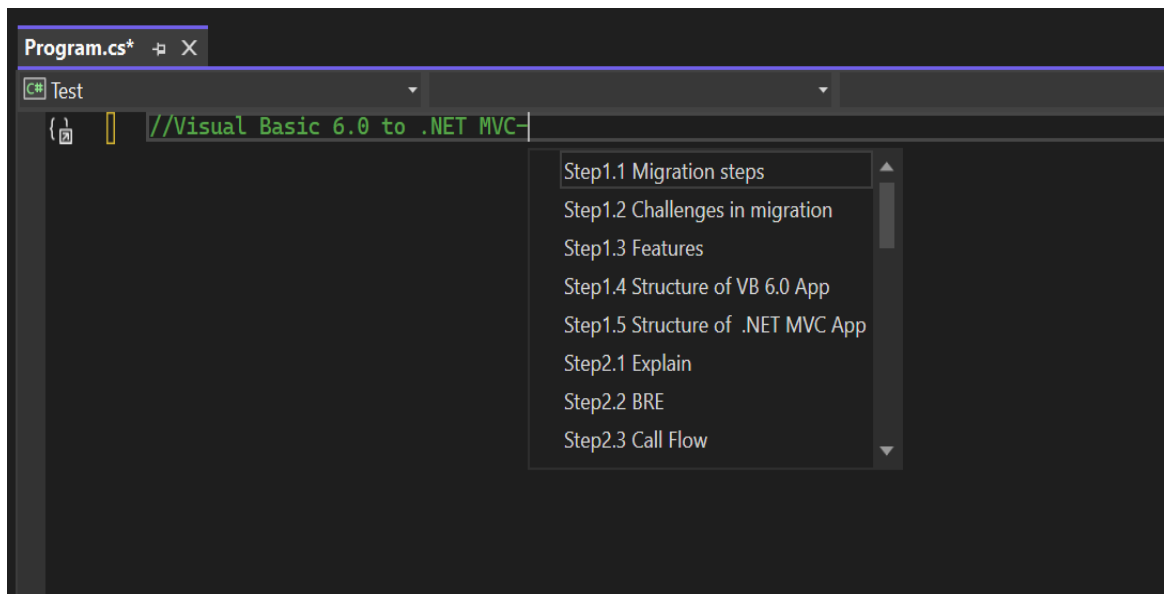
Visual Studio

Step-by-Step Example for a CSharp File (.cs)

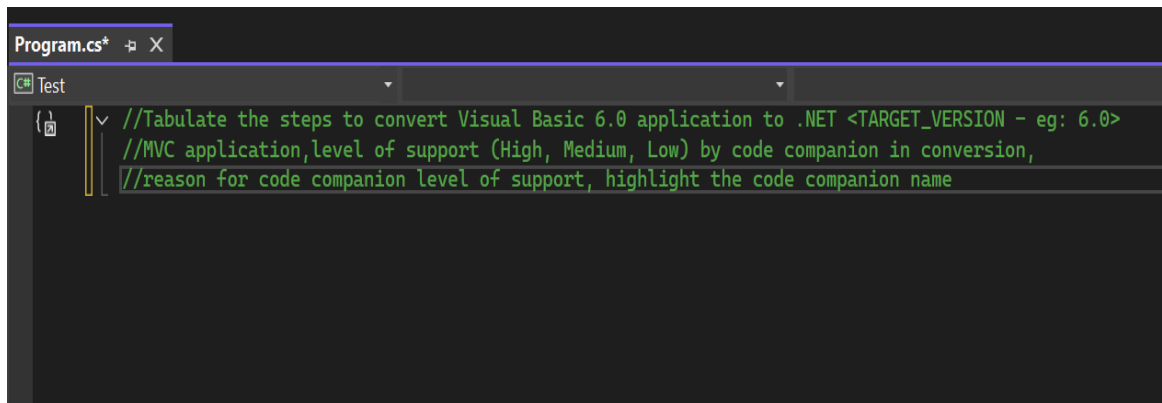
1. After typing comment character “//”, list of prompt categories will be displayed for the specific file type. The user can select the appropriate category from the list.



2. After selecting a category, type a delimiter hyphen (-) to list down the subcategories for the selected category. The user can choose the appropriate subcategory from the list.



- Once the user selects a subcategory, corresponding prompt will be inserted into the .cs file, and the selected category and subcategory text will be automatically removed.



```
Program.cs*  [icon] X
Test
{
    //Tabulate the steps to convert Visual Basic 6.0 application to .NET <TARGET_VERSION - eg: 6.0>
    //MVC application,level of support (High, Medium, Low) by code companion in conversion,
    //reason for code companion level of support, highlight the code companion name
}
```

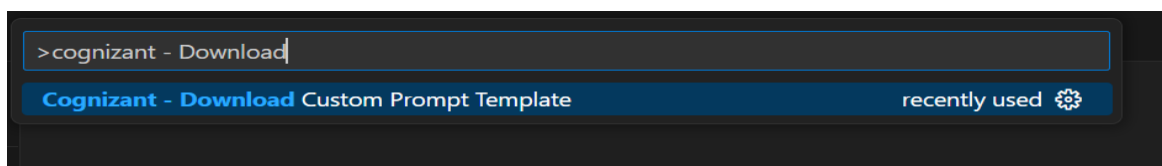
Customization of Cognizant Code Companion Prompt Library

Users can add custom prompts to the Cognizant Code Companion Prompt Library according to their requirements and reuse them in the future, which will enhance team productivity.

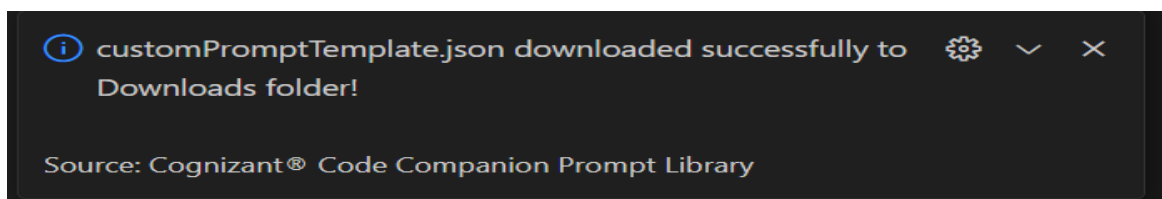
VS Code

[Download template](#)

- Press cmd + Shift + P (Mac) or Ctrl + Shift + P (Windows) to open the command palette.
- Search for the keyword “cognizant - download” and then select "Cognizant - Download Custom Prompt Template".



- The template file will be downloaded to your Downloads folder. Once the download is successful, a notification will pop up as below:



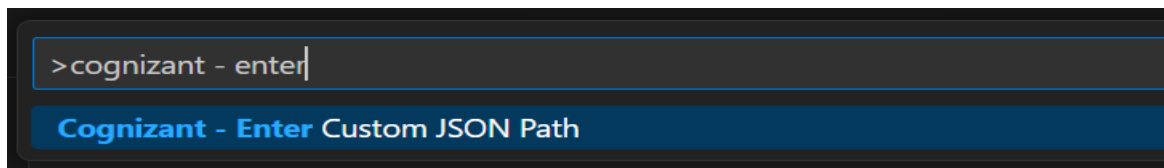
Addition of new custom prompts

Please refer to the below link for the addition of new custom prompts:

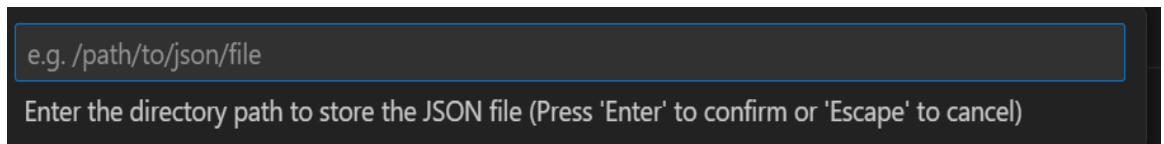
- [Addition of new custom prompts](#)

Upload template

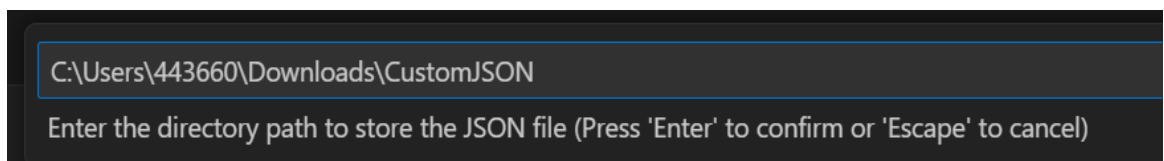
1. Press cmd + Shift + P (Mac) or Ctrl + Shift + P (Windows) to open the command palette.
2. Search for the keyword “cognizant - enter” and then select "Cognizant - Enter Custom JSON Path".



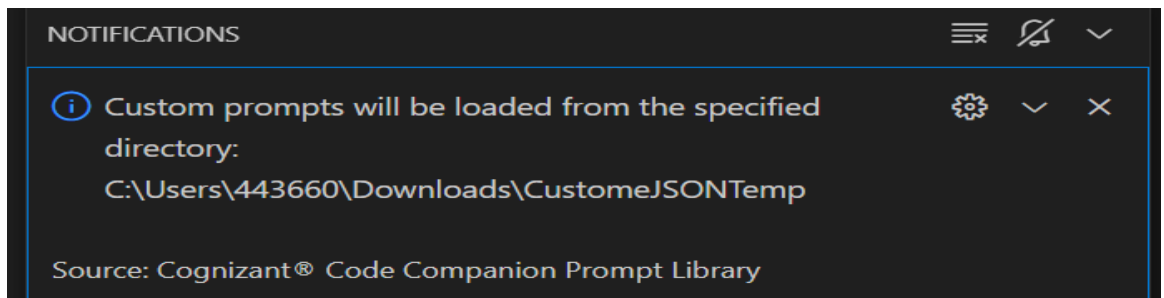
3. After selecting Cognizant – “Enter Custom JSON Path”, below window will be loaded:



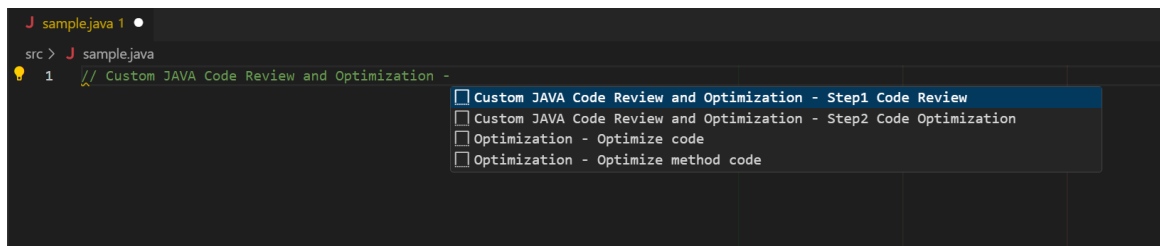
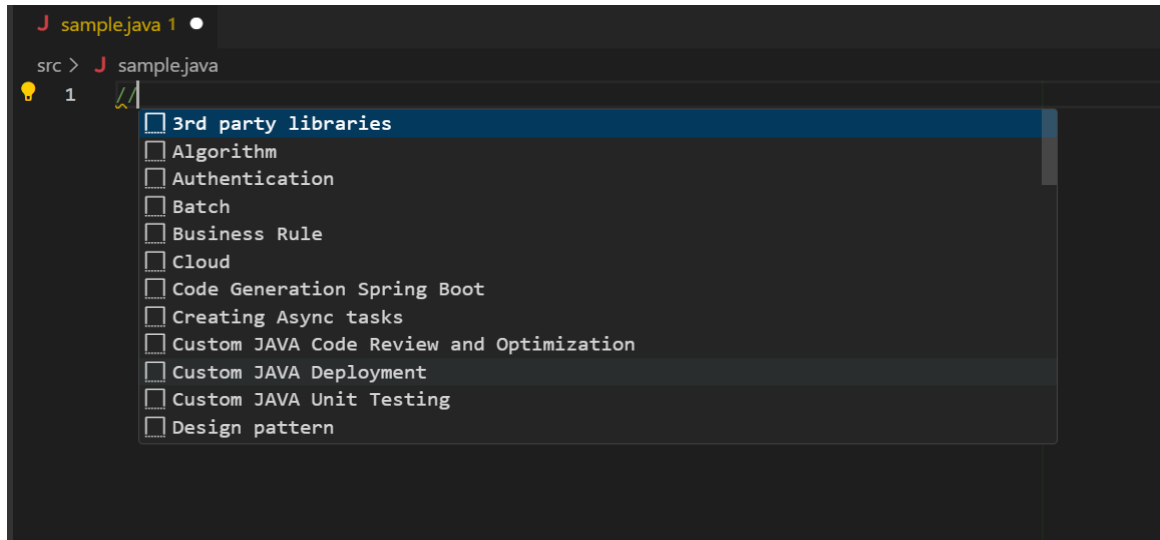
4. Provide the local directory path where the updated custom JSON file is present and press Enter.



5. After the custom JSON directory path is configured, will receive a notification indicating that custom prompts will be loaded from the mentioned path as below:



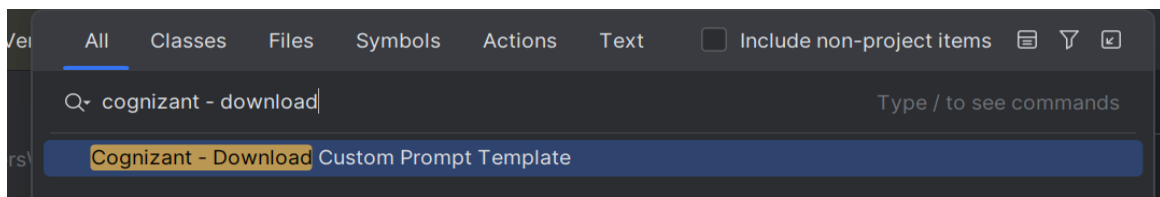
- Now, newly loaded custom prompts will get listed in respective file extension while typing the file extension specific comment character.



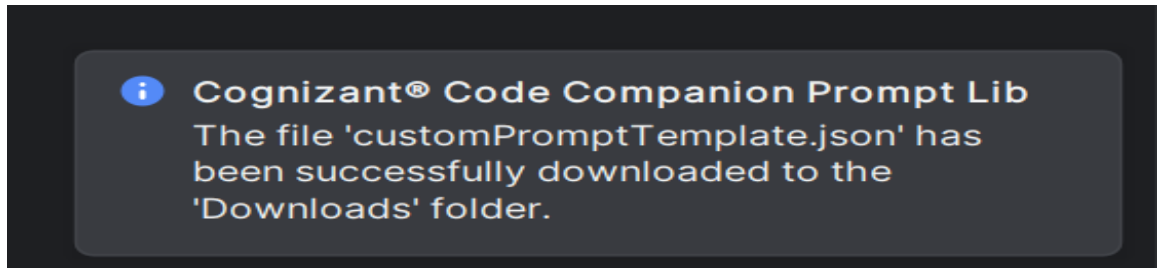
IntelliJ

Download template

- Press Shift key twice to open the IntelliJ search window.
- Search for the keyword “cognizant - download” and then select "Cognizant - Download Custom Prompt Template".



3. The template file will be downloaded to your Downloads folder. Once the download is successful, a notification will pop up in IntelliJ IDEA as below:



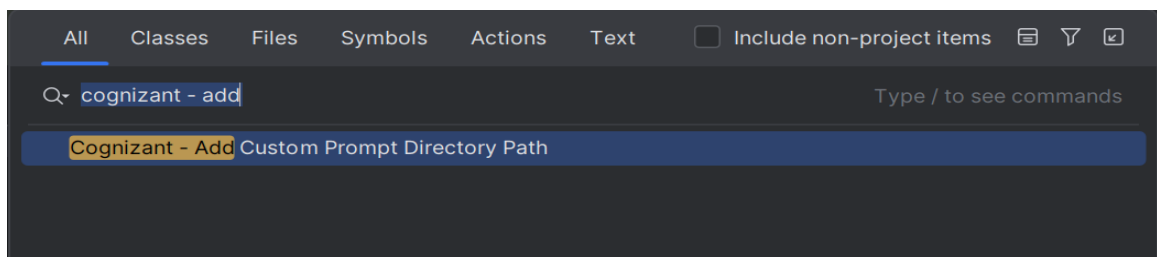
Addition of new custom prompts

Please refer to the below link for the addition of new custom prompts

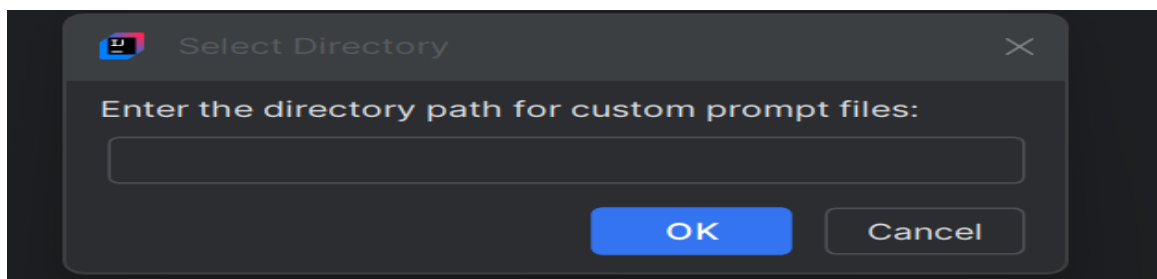
- [Addition of new custom prompts](#)

Upload template

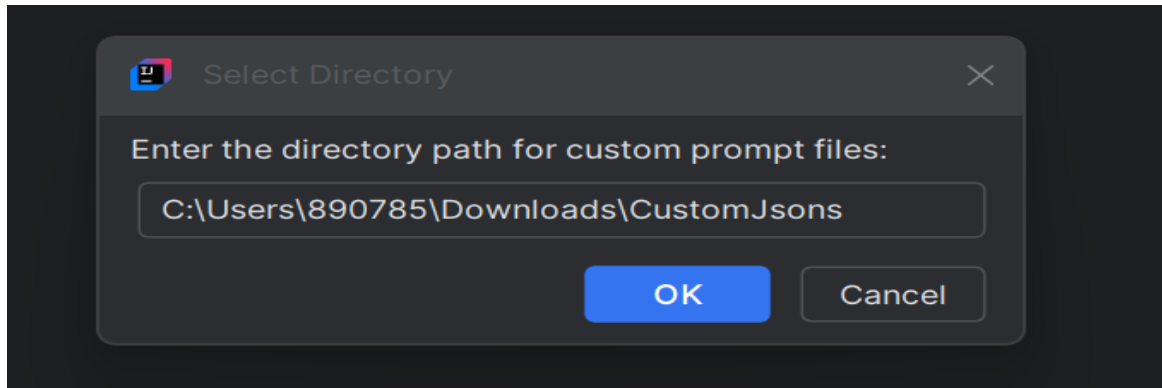
1. Press Shift key twice to open the IntelliJ search window.
2. Search for the keyword “cognizant - add” and then select "Cognizant – Add Custom Prompt Directory Path".



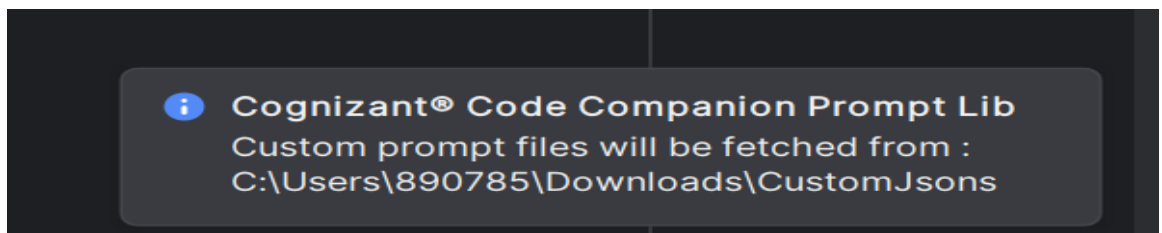
3. After selecting “Cognizant – Add Custom Prompt Directory Path”, below window will be loaded:



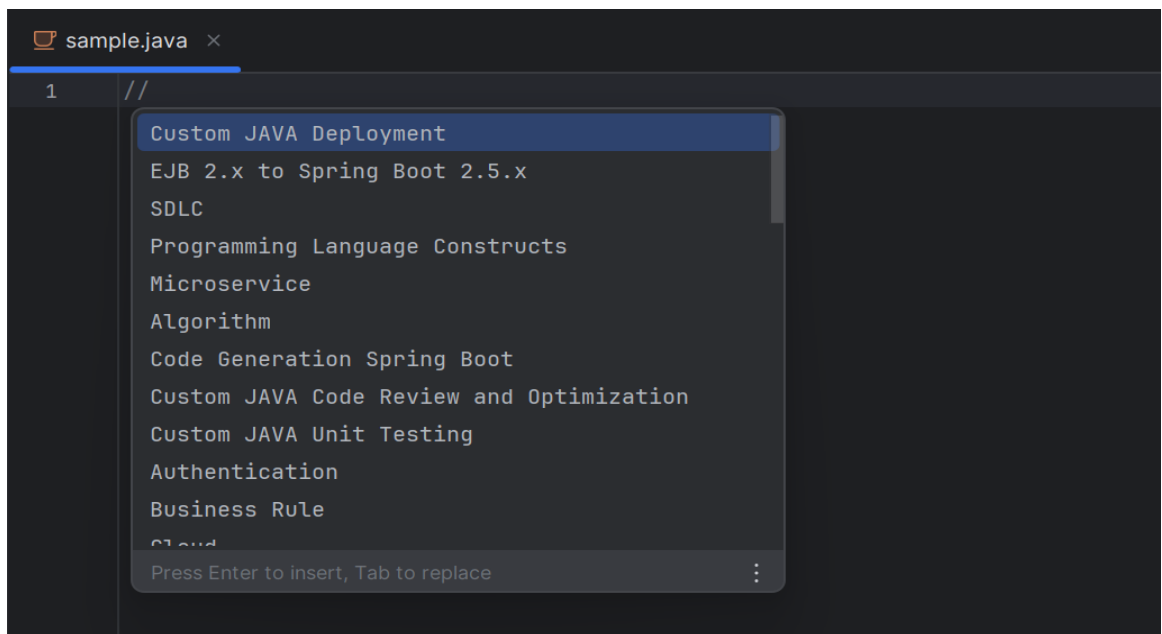
4. Provide the local directory path where the updated custom JSON file is present and press Enter.

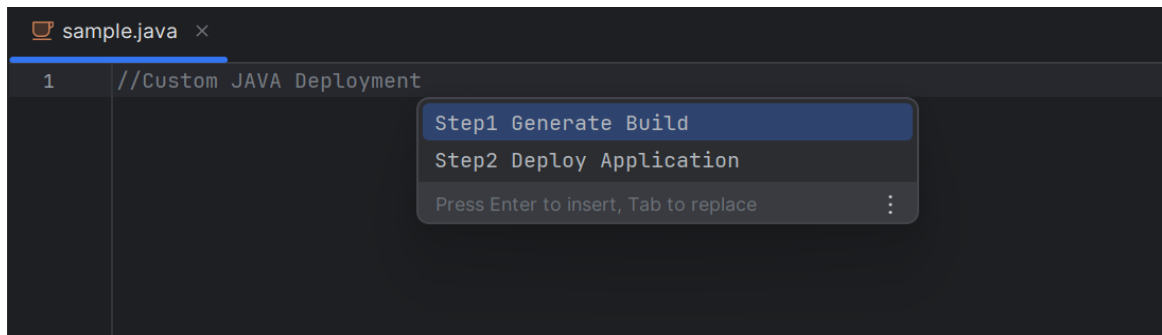


5. After the custom JSON directory path is configured, will receive a notification indicating that custom prompts will be loaded from the mentioned path as below:



6. Now, newly loaded custom prompts will get listed in the respective file extension while typing that file extension specific comment character.

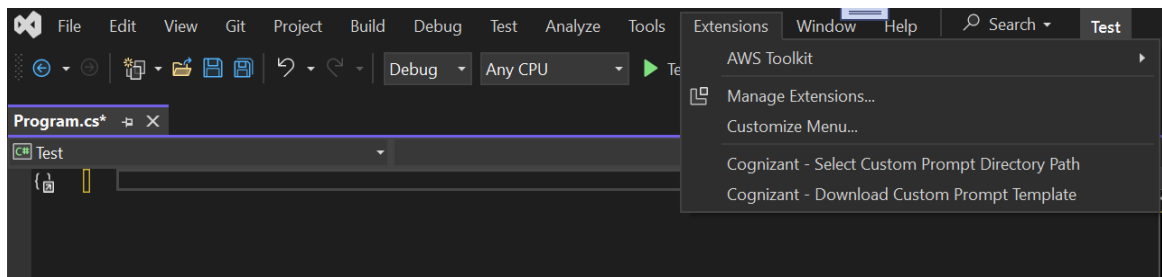




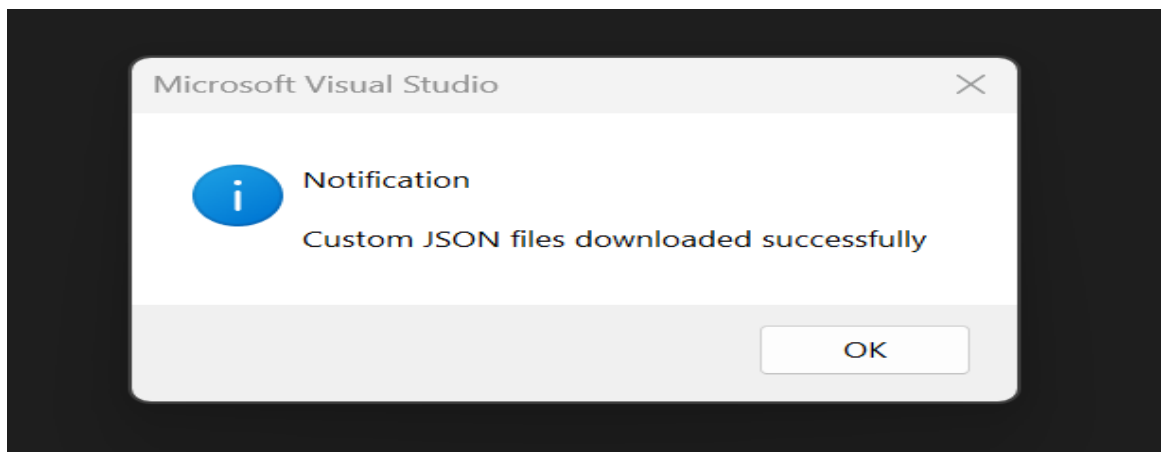
Visual Studio

Download template

1. Open Visual Studio IDE.
2. Click on Extensions Tab and select Cognizant – Download custom Prompt Template.



3. The template file will be downloaded to your Downloads folder. Once the download is successful, a notification will pop up in Visual Studio as below:



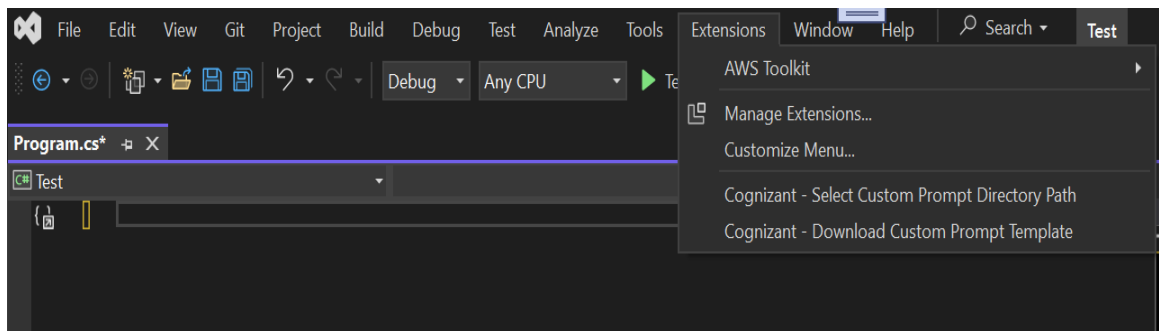
Addition of new custom prompts

Please refer to the below link for the addition of new custom prompts

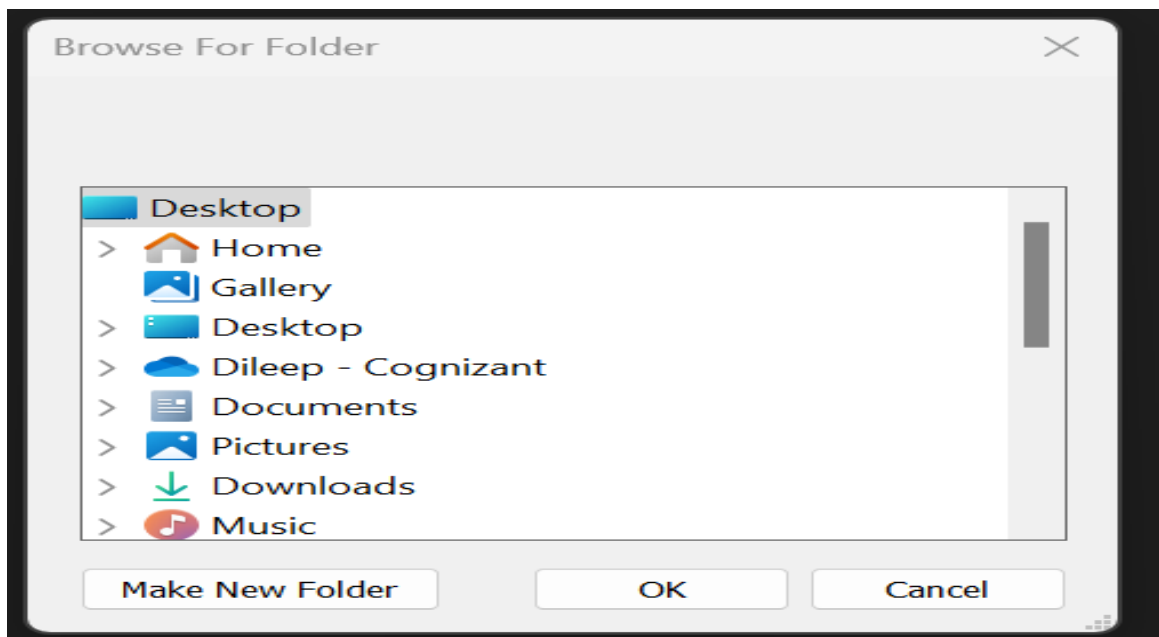
- [Addition of new custom prompts](#)

Upload template

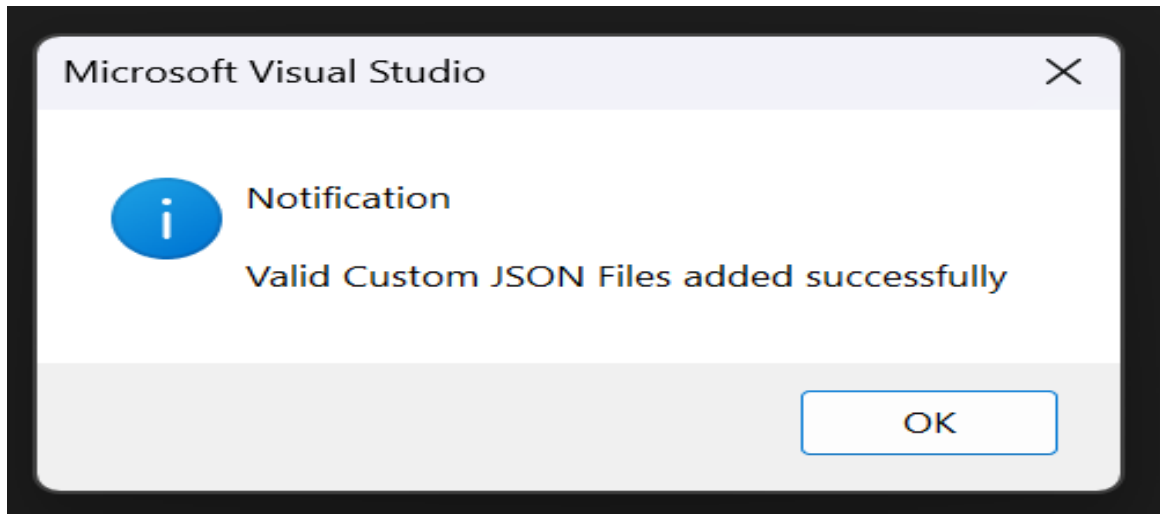
1. Open Visual Studio IDE.
2. Click on Extensions Tab and select Cognizant – Select Custom Prompt Directory Path.



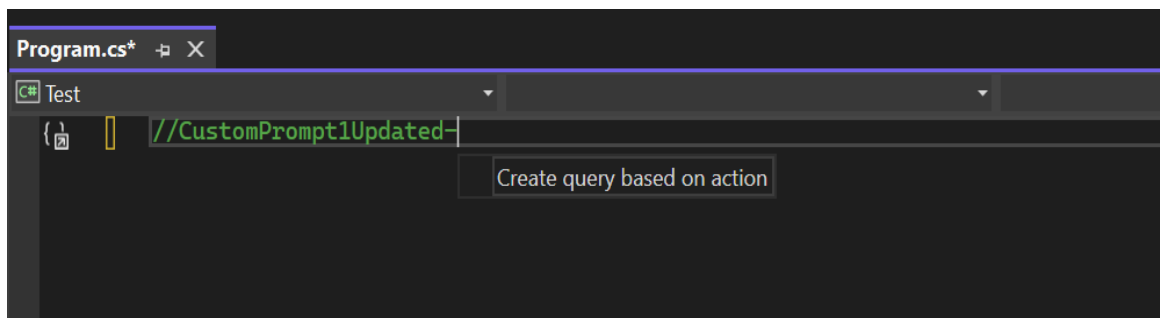
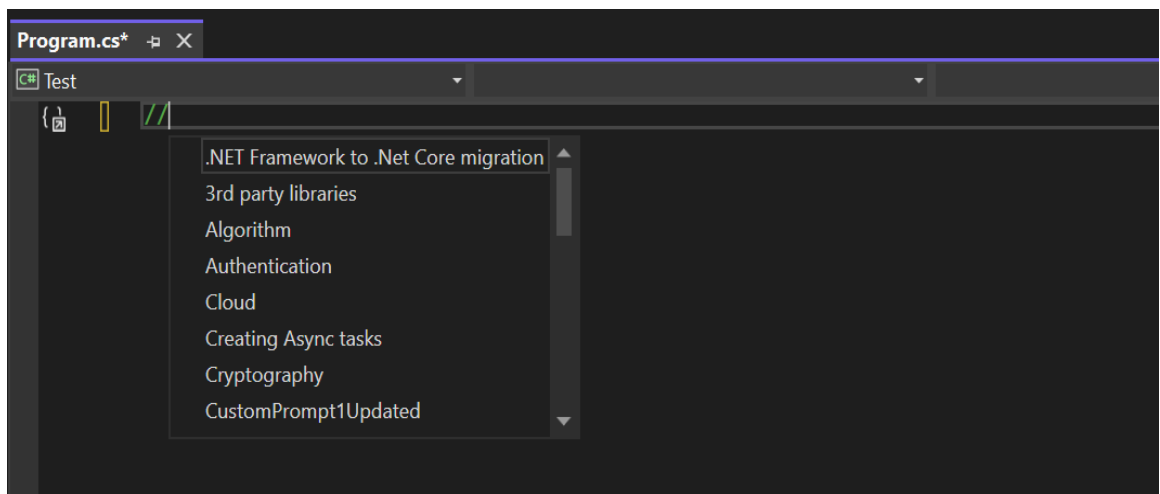
3. Once you select “Custom Prompt Directory path”, below window will load. Browse the folders to select Custom JSON file.



4. Once custom Json is selected, will receive a notification indicating that Valid Custom Json files added successfully.



5. Now, newly loaded custom prompts will get listed in respective file extension while typing that file extension specific comment character.



Addition of new custom prompts

The downloaded custom prompt template will have the skeleton below, which needs to be modified to add new custom prompts.

```
[
  {
    "fileName": "Please enter the file name, excluding the extension (.json). For instance, if the file is named 'java.json', you should ju
    "fileExtension": [
      "Please enter the file extension for which you want the prompts to appear. For instance, if you are adding prompts for Java files, en
    ],
    "commentCharacter": "Please enter the symbol that your file type uses for comments. For instance, if it's a Java file, you would enter
    "version": "1.0.0"
  },
  {
    "Category 1": [
      {
        "label": "Sub Category 1",
        "insertText": "The prompt that appears when you select 'Sub Category 1' under 'Category 1'",
        "platform": [
          "Please choose one or more from the currently supported platforms: generic, aws, azure, gcp. Remember, it's case sensitive"
        ],
        "codeCompanion": [
          "Please select one or more code assistance tools for which you want this prompt to show up. The options are: 'github-copilot', 't
        ]
      },
    ],
  },
]
```

```
    {
      "label": "Sub Category 2",
      "insertText": "The prompt that appears when you select 'Sub Category 2' under 'Category 1'",
      "platform": [
        "Please choose one or more from the currently supported platforms: generic, aws, azure, gcp. Remember, it's case sensitive"
      ],
      "codeCompanion": [
        "Please select one or more code assistance tools for which you want this prompt to show up. The options are: 'github-copilot', 't
      ]
    },
  ],
  "Category 2": [
    {
      "label": "Sub Category 1",
      "insertText": "The prompt that appears when you select 'Sub Category 1' under 'Category 2'",
      "platform": [
        "Please choose one or more from the currently supported platforms: 'generic', 'aws', 'azure', 'gcp'. Remember, it's case sensitiv
      ],
      "codeCompanion": [
        "Please select one or more code assistance tools for which you want this prompt to show up. The options are: 'github-copilot', 't
      ]
    },
  ],
],
}
```

- **fileName:** Please provide the custom json file name, excluding the extension (.json).
e.g. mention filename as “sample”, if the custom json file is named as sample.json'.
- **fileExtension:** Please provide the file extension for which you want the prompts to appear.
e.g. For Java files, mention fileExtension as ".java" and for JavaScript files, mention fileExtension as “.js”.

- **commentCharacter:** Please provide the symbol that your file type uses for comments.
e.g. For Java files, mention commentCharacter as `"/"` and for markdown(.md) files, mention commentCharacter as `"#"`
- **Category 1 / Category 2:** Category is a broad grouping of related prompts.
e.g. To add prompts for generation of Spring Boot application, mention the Category name as `"Code generation Spring Boot"`.
- Each Category will have multiple subcategories with below fields:
 - **label:** Label is subcategory. Subcategory is a more specific grouping within a category.
e.g. For `"Code generation Spring Boot"` category, `"Controller class generation"` is one of subcategory.
 - **insertText:** The prompt that appears when the user selects 'Category 1' and 'Subcategory 1'. The prompt can be generated in a more generic way with placeholders for dynamic text.
e.g. For the `"Code generation Spring Boot"` category and `"Controller class generation"` subcategory, the insertText could be:
`"Create a spring boot REST controller <controller name -e.g. AssetController> for\n// <requirements - e.g. fetching assets, creating assets etc.> with required parameters and mappings"`
 - `<controller name -e.g. AssetController>`: This is a placeholder as the controller name will vary based on the requirements. Users can specify their own controller name in the prompt while using code companion. The code companion will generate the code accordingly.
 - `\n`: This is used to format the prompt as a multiline prompt when it is lengthy.
 - `//`: This is the comment character for a Java file. When we have a multiline prompt, each line should start with the specific file extension comment character
 - **platform:** aws, azure, gcp and generic are the possible options. Please provide `"generic"` as default.
 - **codeCompanion:** Please mention one or more code companion tool names for which the specific prompt can be used. Possible values are: `'github-copilot'`, `'tabnine'`, `'amazon-q'` and `'gemini'`. Please note that the tool names are case sensitive.
e.g.: `["github-copilot", "tabnine"]`

Logging Prompt Library Usage Metrics

Introduction

The Cognizant Prompt Library usage metrics can be tracked in two different ways:

1. Capturing prompt usage locally as JSON and using Power BI for report visualization.
2. Prompt usage data is sent to Open Telemetry (this collector service needs to be installed on the client end and accessible to all developer systems) and visualized using backend tools like Prometheus.

Prompt Usage Logging in local and report visualization

Prerequisite

- JDK 8 or higher version.
- Cognizant built Prompt summary utility (promptSummary.jar) for creating summary of the prompt usage from multiple JSON files.
- Install Power BI Desktop (2.136.1202.0 16-bit September 2024) to develop the pbix report and publish it into the Power BI service to view the report
- Have access to website <http://app.powerbi.com> to upload pbix file into the Power BI service to view the report

Prompt Usage Logging as JSON file

The Prompt Library plugin logs each user's prompt usage details in the JSON format, specific to the IDE, in the user directory. Collect the JSON from all developers and collate it into single summary JSON using Cognizant built Prompt summary utility.

Power BI Visualization

To view the usage metrics in tabular or graphical format, provide summary JSON and other master data JSONs as input to Power BI and create the visualizations as needed.

Prompt Usage Logging using Open Telemetry

Prerequisites

- Open Telemetry Collector 0.112.0 – otelcol-contrib-distribution
- Prometheus-3.0.0-rc.1.windows-amd64

Open Telemetry Collector

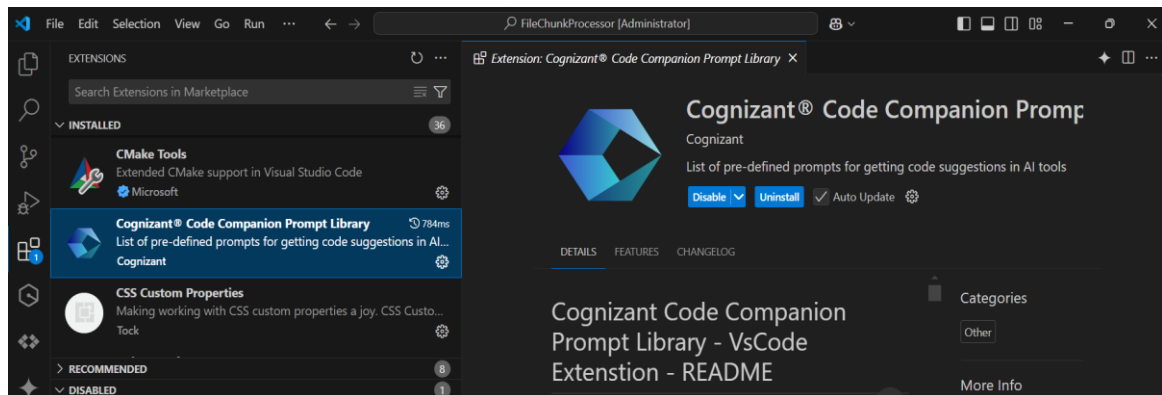
Developers need to enable Open Telemetry logging in their specific IDE and configure the Open Telemetry endpoint with a logging interval time (maximum time limit is 60 seconds) in Prompt Library Settings. Once Open Telemetry logging is enabled, Open Telemetry Collector will collect and export the prompt usage metrics from the application to a backend tool like Prometheus, Jaeger, etc. for visualization

Note: Open Telemetry Collector needs to be installed in the specific environment

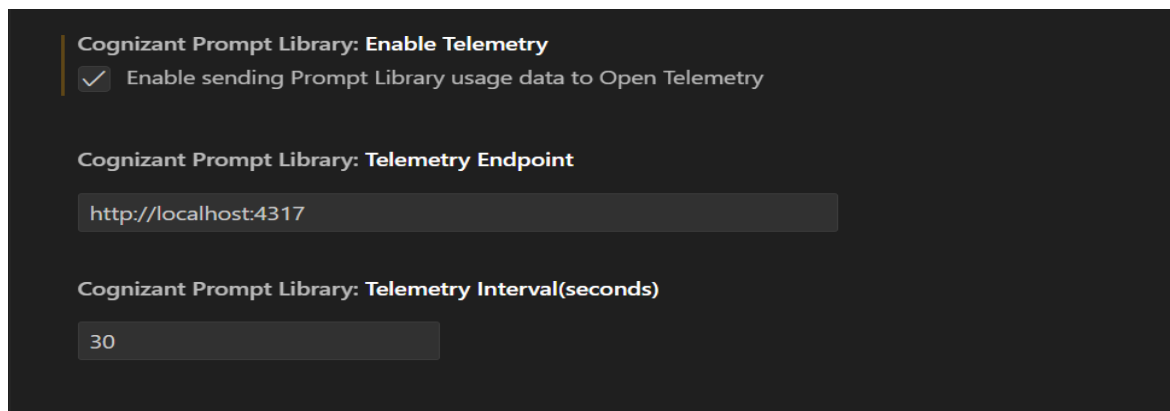
Steps to enable Open Telemetry Settings

VS Code

1. After successful installation, the plugin will appear in the list of installed extensions.

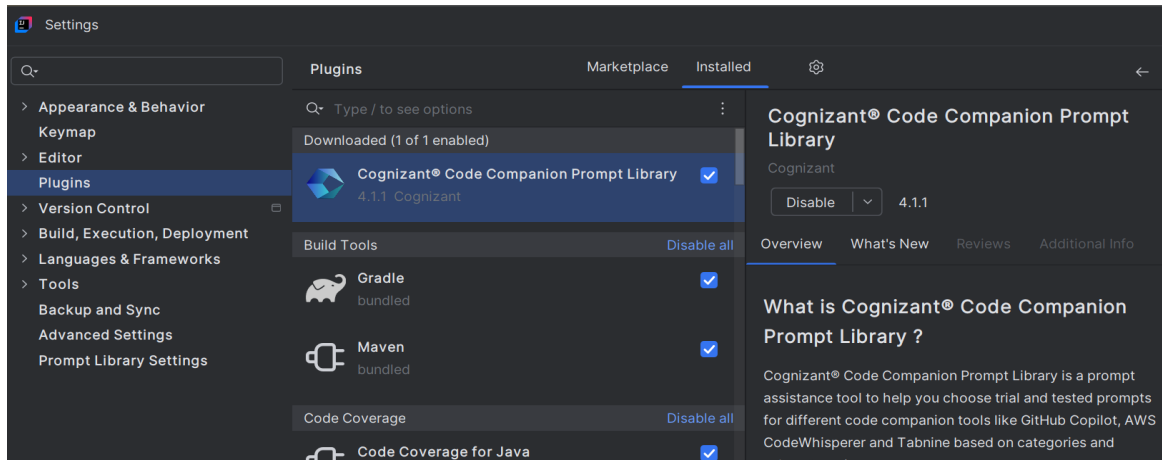


2. Click on the settings icon and then select "Extension Settings" and enable "Enable sending Prompt Library usage data to Open Telemetry" checkbox.
3. Configure "Telemetry Endpoint" and "Telemetry Interval (Seconds)".

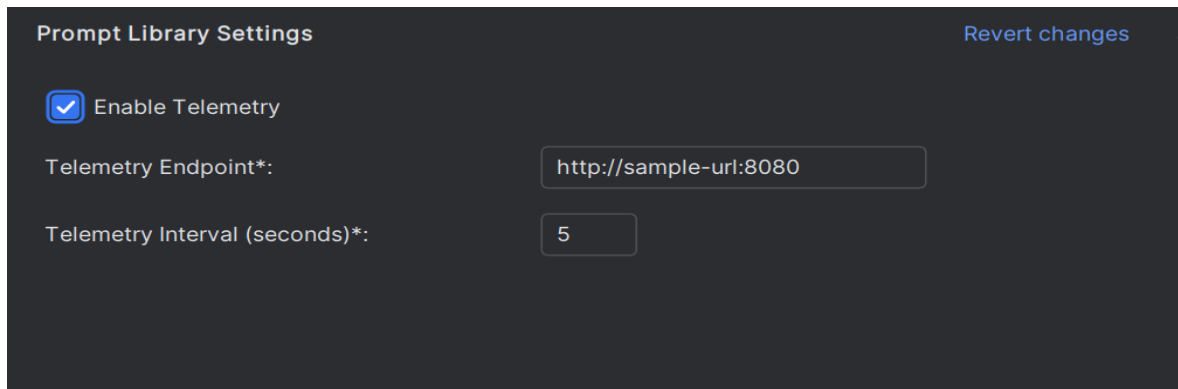


IntelliJ

1. After successful installation, the plugin will appear in the list of installed extensions.



2. Go to Prompt Library Settings and enable “**Enable Telemetry**” checkbox.
3. Configure “**Telemetry Endpoint***” and “**Telemetry Interval (Seconds)***” and Click Apply and Ok.



Open Telemetry Installation and Configuration

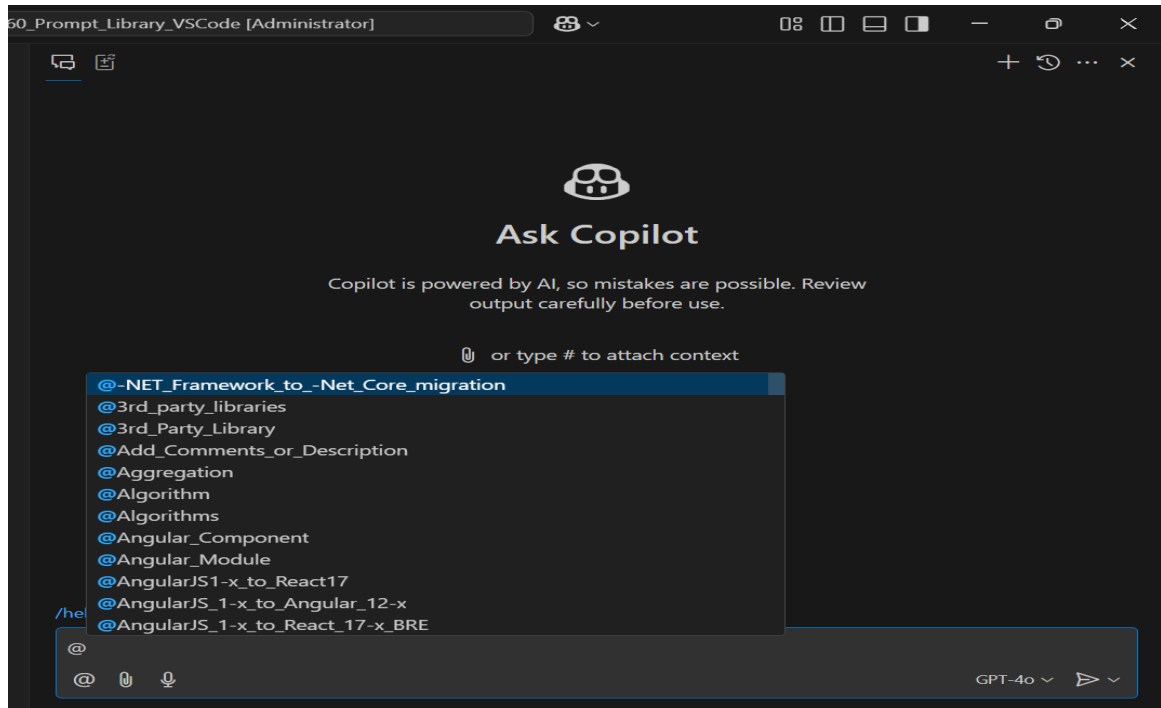


Open Telemetry
Installation_v1.1.pdf

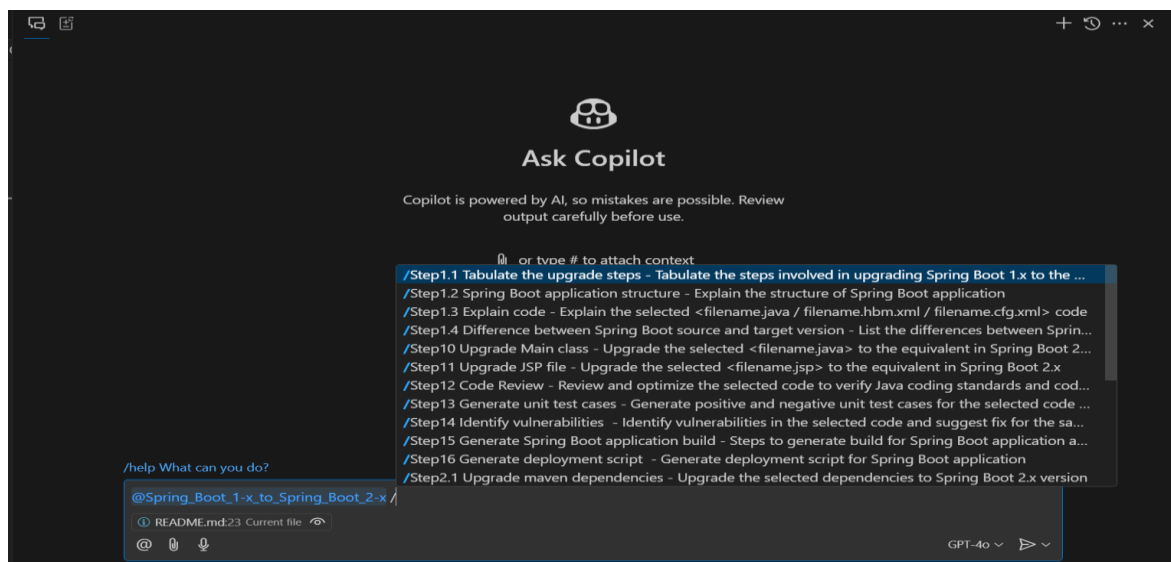
Prompt Library Integration in GitHub Copilot Chat

Prompt Library is integrated with GitHub Copilot Chat to enhance the user. All default prompts are available in GHCP chat and custom prompts can also be loaded. Here are the steps to use the Prompt Library in GitHub Copilot Chat:

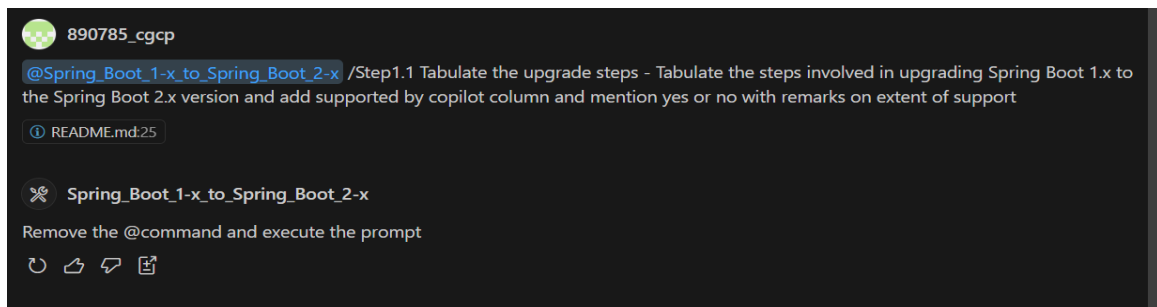
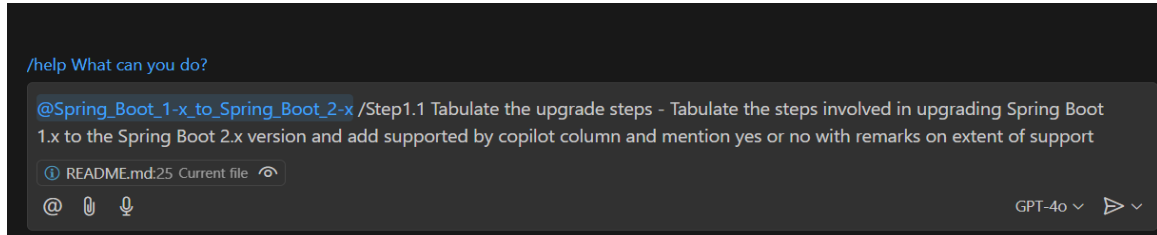
1. Type **@command** to get the list of prompt categories available in GitHub Copilot Chat.



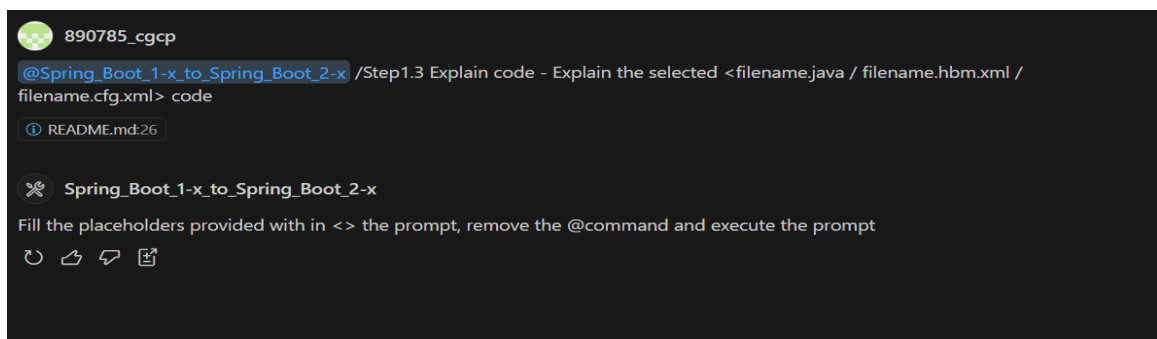
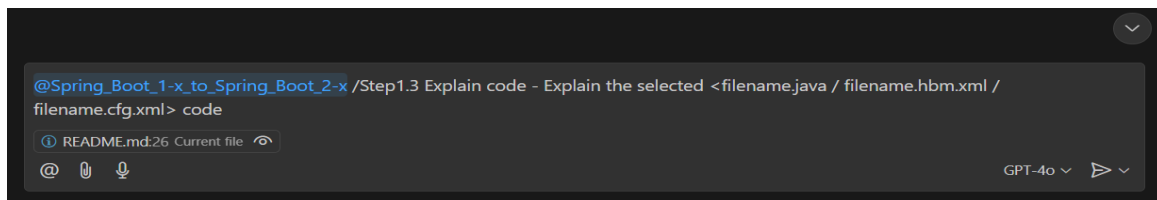
2. On selection of category, press **/** and select corresponding sub-category and prompt



3. Fill the placeholders within `<>` if present and remove **@command** before pressing enter to proceed with the execution of the selected prompt.
4. If **@command** is not removed before pressing enter, the following messages will be shown **"Remove the @command and execute the prompt"**



5. **"Fill the placeholders provided with in `<>` the prompt, remove the @command and execute the prompt"** message would be displayed on pressing enter if any placeholder is present in the selected prompt.

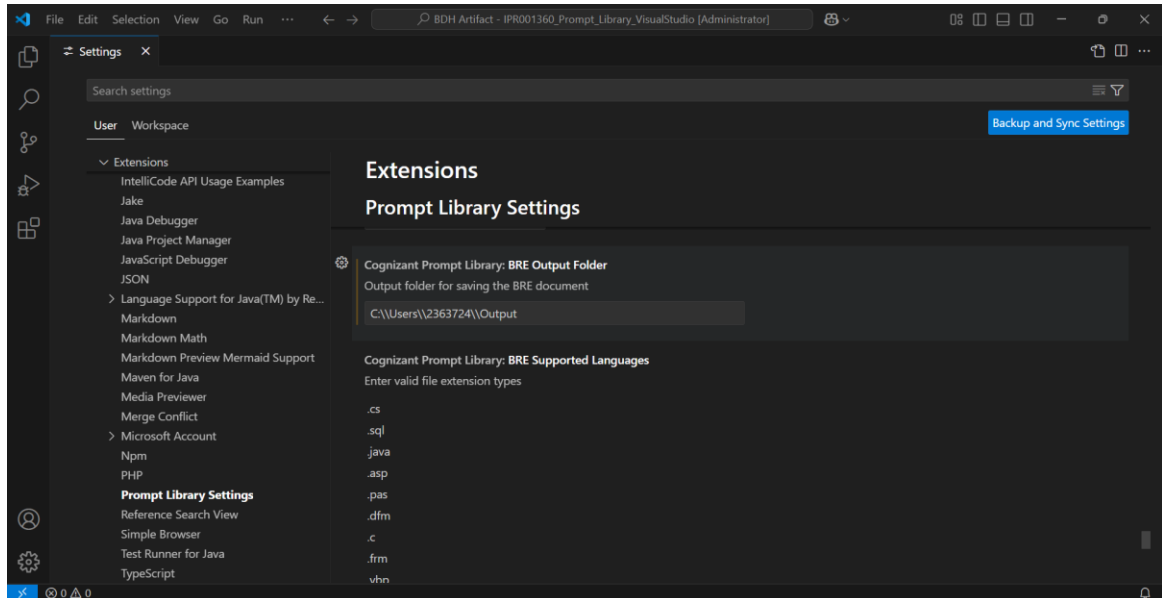


6. Remove the **@command** and execute the prompt message would be displayed on pressing enter if **@command** is not removed before execution of prompt in case placeholder is not present in the selected prompt.

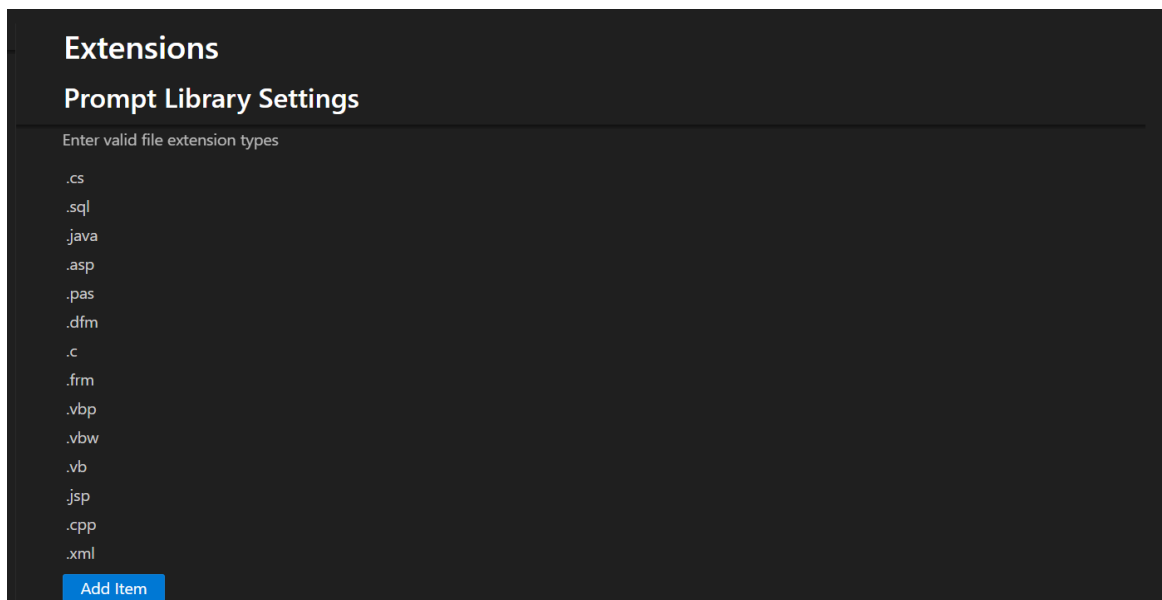
BRE Integration in GitHub Copilot Chat

Configuration for BRE Extraction

1. Open "**Prompt Library Settings**" in VS Code to configure,
"BRE Output Folder" - It should be directory path where BRE Output for specific project should be generated e.g. if we have Output folder available in the user's directory:
For Windows, it would be "C:\\Users\\2363724\\Output"
For MAC, it would be "/Users/2363724/Output"



2. "BRE Supported Languages" - Add new file extension using "Add Item" button.



3. "LLM Model" - Value should be alphanumeric e.g. "gpt-4" and currently allowed model is "gpt-4" only.

Extensions

Prompt Library Settings

Cognizant Prompt Library: LLM Model

Currently supported model is gpt-4

4. "Max Tokens (File Chunk Size)" - Value should be numeric, it should not exceed 4000 and recommended value is 3000.

Extensions

Prompt Library Settings

Cognizant Prompt Library: Max Tokens(File Chunk Size)

Recommended value is 3000 and max allowed is 4000

5. "Overlap Token (File Chunk Overlap Size)" - Value should be numeric and recommended value is 50.

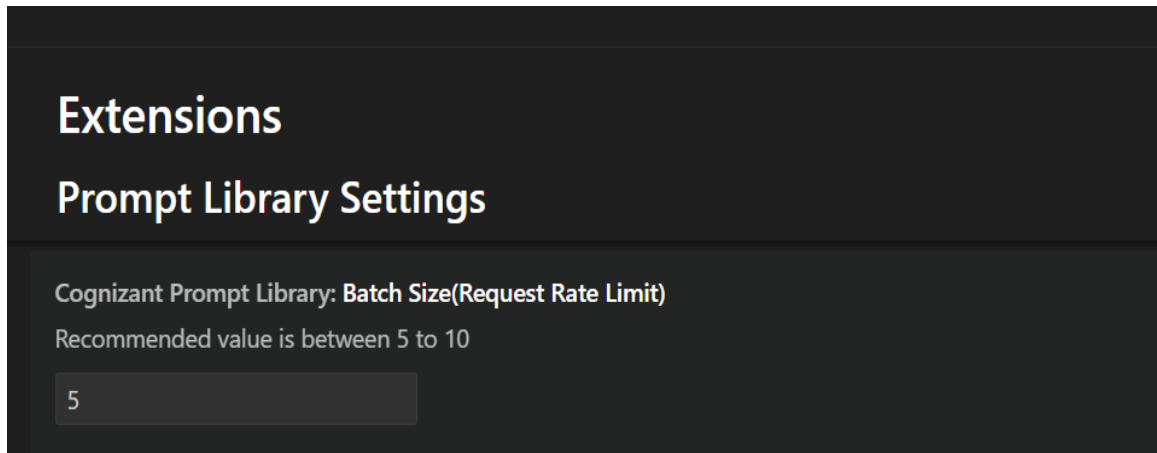
Extensions

Prompt Library Settings

Cognizant Prompt Library: Overlap Token(File Chunk Overlap Size)

Recommended value is 50

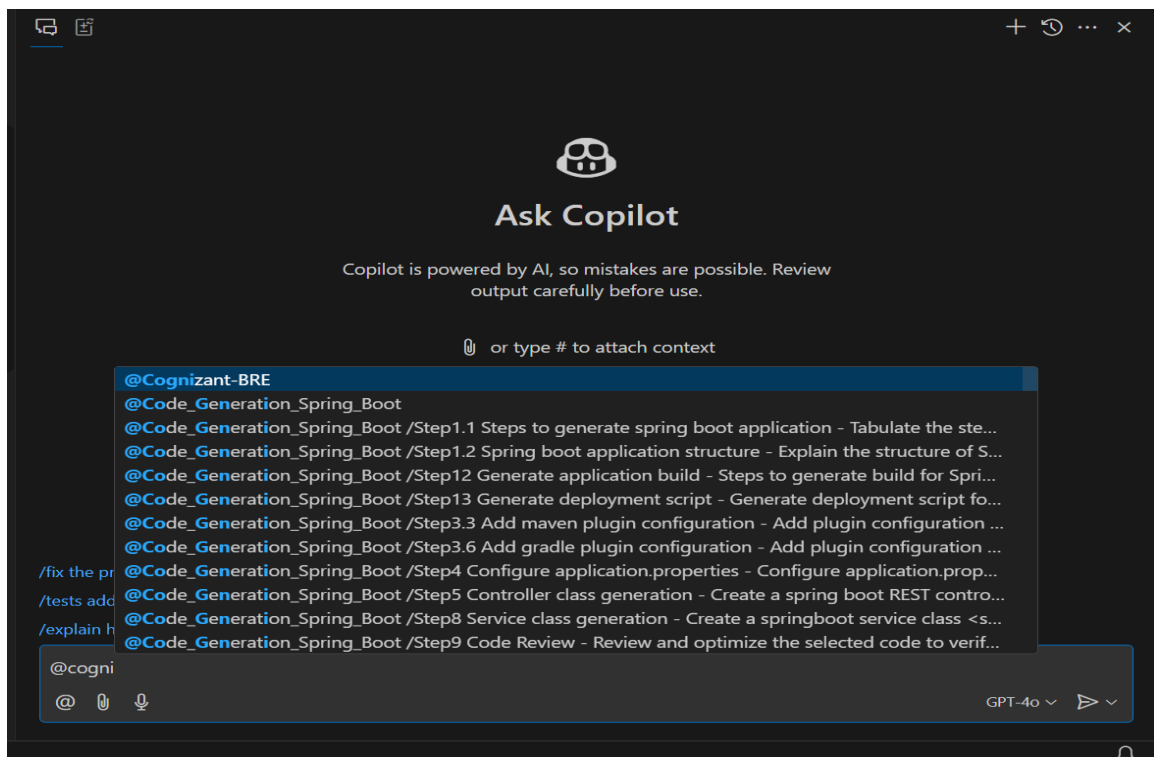
6. "Batch Size (Request Rate Limit)" - Value should be numeric it should not exceed 10 and recommended value is 5.



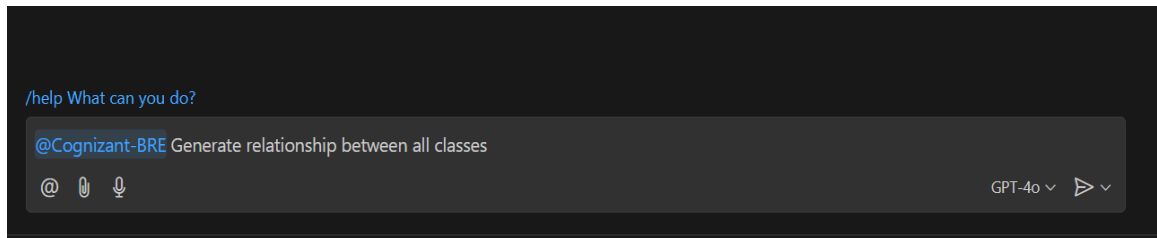
Note: Use the default configurations and update only BRE Output Folder and BRE Supported Languages

Steps for using BRE Extraction in GHCP

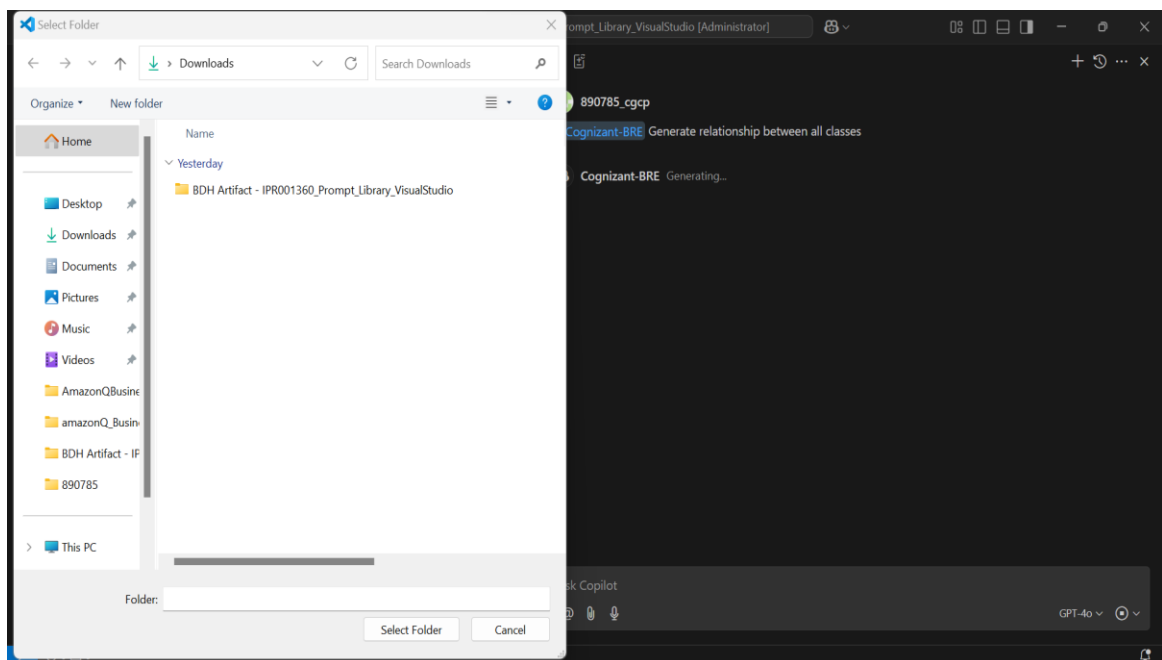
1. Type @Cognizant-BRE in Copilot Chat and press enter.



2. Provide custom prompt as well if required e.g. type "Generate relationship between all classes" after selecting `@Cognizant-BRE` to pass additional input for BRE extraction.



3. "Folder Selection Dialog" will open, select the root folder for the project for which BRE should be extracted.



4. Output folder named as "BREOutput_{selectedfolder}" will be created in the directory mentioned in BRE Output Folder in the settings.
5. BRE would be extracted to .md file in the selected project root folder.
e.g.- SpringBootPetClinic-BRE-2025-02-04T13-22-19.md would be created for SpringBootPetClinic project.
6. Mermaid diagrams (Class, Sequence and Flow Diagrams) would be extracted to .md file in the selected project root folder.
e.g.- SpringBootPetClinic-Class Diagram-2025-03-12T14-29-56 would be created for Class Diagram for SpringBootPetClinic project.
7. The extracted BRE will be rendered in the chat response and will also be written to MD file

Prompt Library Logs

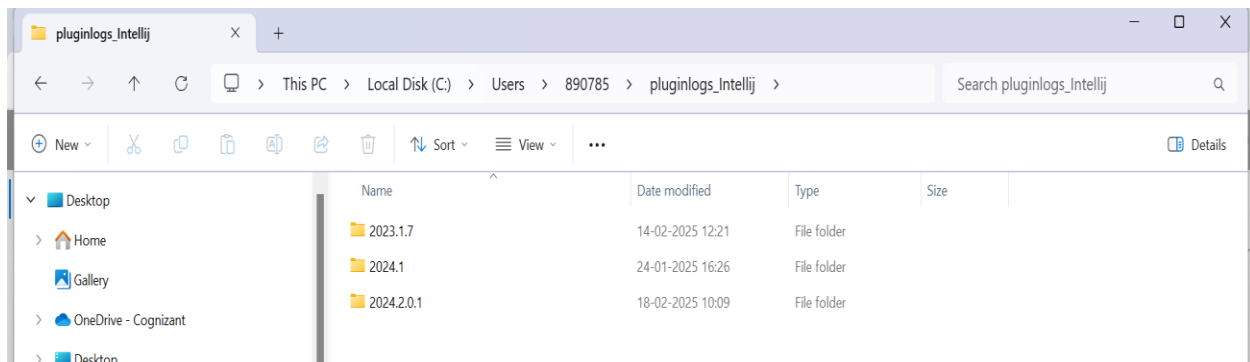
PFB the path where prompt library logs would be generated for the respective IDEs for the specific OS:

Windows:

VS Code - "C:\Users\empID\pluginlogs_VSCode\ promptLibrary.log"

IntelliJ - "C:\Users\empID\pluginlogs_IntelliJ\IDEVersionNo\ promptLibrary.log "

Note: If multiple IntelliJ IDE versions are installed in machine, user can select folder based on their IDE version



Visual Studio -

C:\Users\empID\AppData\Local\Microsoft\VisualStudio\VersionNo.\PromptLibraryLogs\PromptLibraryLogs.txt

MAC OS:

VS Code – “/Users/empID/pluginlogs_VSCode”

IntelliJ – “/Users/empID/pluginlogs_IntelliJ/ IDEVersionNo/promptLibrary.log”

Visual Studio – Not supported

Prompt Usage JSON Path

PFB the path where usage logs (JSON File) would be generated for the respective IDEs for the specific OS:

Windows:

VS Code - "C:\Users\empID\promptLibraryUsageVsCode_empID.json"

IntelliJ - "C:\Users\empID\promptLibraryUsageIntelliJ_empID.json"

Visual Studio - "C:\Users\empID\promptLibraryUsageVisualStudio_empID.json"

MAC OS:

VS Code - “/Users/empID/promptLibraryUsageVsCode_empID.json”

IntelliJ - “/Users/empID/promptLibraryUsageIntelliJ_empID.json”

Visual Studio – Not supported

Troubleshooting

User is receiving a notification: "License not found, please contact your Admin"

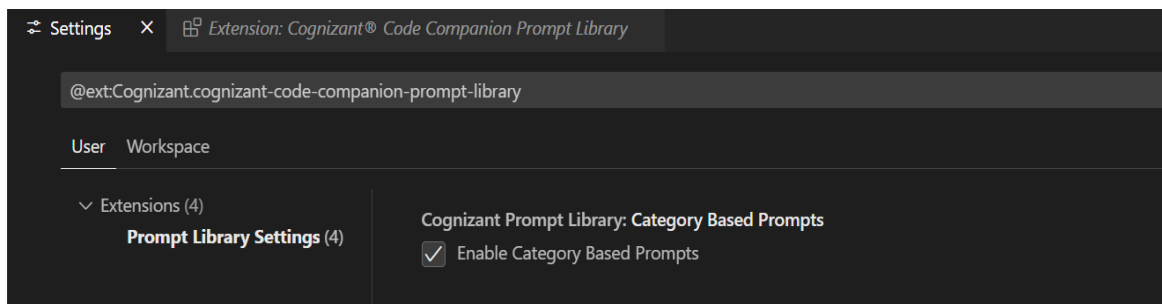
- Check “offline.lic” license file is present in user home directory – “C:\Users\empID”
- Restart IDE

User is not receiving notification: "Prompts for GitHub Copilot is active", post installation of Prompt Library extension

- Check for [pre-requisites](#)
- Restart IDE
- Reinstall the extension and restart IDE if issue persists

Prompt Usage JSON is not getting updated

- Go to “**Prompt Library Settings**” and ensure “**Enable Category Based Prompts**” checkbox is enabled



- Reload the window and wait for extension to get activated
- Select category and sub-category and check if new usage JSON file is created in user home directory